

UNIVERSITÀ DEGLI STUDI DI MILANO
FACOLTÀ DI SCIENZE E TECNOLOGIE

DIPARTIMENTO DI INFORMATICA
“GIOVANNI DEGLI ANTONI”



LAUREA MAGISTRALE IN
SICUREZZA INFORMATICA

TODO: TITOLO DELLA TESI

Autore: Enea Manzi
Matricola: 65935A

Relatore: Prof. Marco Anisetti
Correlatore: Prof. Claudio Agostino Ardagna

A.A. 2025-2026

Abstract

Ringraziamenti

Indice

Abstract	ii
Ringraziamenti	iii
Elenco delle figure	vi
Elenco delle tabelle	vii
Elenco dei codici	viii
1 Introduzione	1
1.1 Motivazione e gaps	1
1.2 Obiettivi	1
1.3 Struttura della tesi	1
2 Background e Stato dell'Arte	2
2.1 Esempio di sezione	2
3 Metodologia, Architettura e Principi di Design	3
3.1 Principi Fondamentali: il Perché delle Scelte Architettureali	3
3.1.1 Agnosticismo Applicativo (D1.P1)	4
3.1.2 La Specifica OpenAPI come Unico Oracolo Formale (D3.P2)	5
3.1.3 Config-Driven Development (D3.P1)	6
3.1.4 Riproducibilità Deterministica (D3.P4)	7
3.2 Architettura dell'Assessment Engine	7
3.2.1 Flusso di Dipendenze Monodirezionale (D1.P2)	8
3.2.2 Scheduling Basato su DAG e Orchestrazione Deterministica (D2.P2)	8
3.2.3 Split State e Immutabilità (D1.P3)	9
3.2.4 Gateway-Agnostic Adapter (D1.P6)	11
3.2.5 Streaming Evidence Store (D4.P1)	11
3.2.6 Assenza di Dipendenze di Stato Esterno (D3.P5)	12
3.3 Domini di Sicurezza e Box Gradient Applicato	12
3.3.1 Tassonomia degli Otto Domini di Assurance	13
3.3.2 Il Box Gradient come Mapping dalle Precondizioni al Privilegio (D3.P3)	14

4	Implementazione	17
4.1	La Pipeline di Esecuzione a Sette Fasi e il Core Engine	17
4.1.1	Fail-Safe Error Isolation (D4.P3)	19
4.1.2	Gerarchia delle Eccezioni con Phase Mapping (D4.P4)	19
4.2	Discovery Dinamico e Risoluzione del Grafo	20
4.2.1	Discovery Dinamico via <code>pkgutil</code> (D2.P1)	20
4.2.2	Scheduling Topologico e <code>DAGScheduler</code> (D2.P2)	21
4.2.3	Invarianti di <code>TestResult</code> a Livello di Tipo (D4.P9)	22
4.3	Architettura dei Test Nativi e Contratto <code>BaseTest</code>	22
4.3.1	Gerarchia Duale <code>BaseTest</code> / <code>ExternalToolTest</code> (D1.P4)	23
4.3.2	Auth Abstraction Layer e Idempotenza dei Token (D2.P5)	24
4.3.3	Tracciabilità Normativa e Distinzione Finding/InfoNote (D5.P1, D5.P2)	24
4.3.4	Safe HTTP Probing Policy e Oracle a Tre Esiti (D4.P7)	25
4.4	Gerarchia dei Connettori e Integrazione Tool Esterni	26
4.4.1	Gerarchia a Tre Livelli e Separazione Dati/Valutazione (D1.P5, D2.P3)	26
4.4.2	Lifecycle dei Connector e Dependency Injection (D2.P4)	27
4.4.3	Classificazione A/B, Path Seed e Catalogo dei Payload (D2.P6, D2.P7, D2.P8)	28
4.4.4	Isolamento della Dipendenza AGPL (D7.P2)	29
4.5	Implementazione del Gateway Adapter e Teardown LIFO	29
4.5.1	<code>KongGatewayAdapter</code> : Accesso Read-Only all'Admin API	29
4.5.2	Degradazione Controllata a Tre Livelli (D4.P5)	30
4.5.3	Teardown Best-Effort in Ordine LIFO (D4.P6)	31
4.5.4	Astrazione Trasparente dell'URL di Deployment (D7.P3)	32
4.6	Evidence Store e Sanitizzazione delle Credenziali	32
4.6.1	Ciclo di Vita dello Streaming Evidence Store (D4.P1)	32
4.6.2	Sanitizzazione Centralizzata delle Credenziali (D4.P2)	33
4.6.3	Dual Audit Trail e Report Domain-Centric (D5.P5)	34
5	Validazione sperimentale e risultati	37
5.1	Topologia del Testbed Sperimentale	37
5.1.1	Architettura del Testbed	38
5.1.2	Configurazione Intenzionale per la Copertura dei Finding	39
5.1.3	Struttura RBAC e Provisioning degli Utenti	40
5.2	Release-Readiness e Integrità Architetturale	40
5.2.1	Analisi Statica del Codebase	41
5.2.2	Dual-Layer Type Safety (D6.P3)	41
5.2.3	Indicatori di Qualità dell'Ingegneria di Produzione	42
5.3	Copertura Funzionale e Idempotenza dell'Assessment	43
5.3.1	Analisi dei Risultati per Dominio	44
5.3.2	Idempotenza e Validazione del Teardown	45

Indice

5.4	Footprint Computazionale e Benchmarking del DAG	46
5.4.1	Regime di Esecuzione: IO-Bound, non CPU-Bound	46
5.4.2	Topologia del DAG e Implicazioni sull'Ordinamento dell'Esecuzione	47
	Sintesi della Validazione	48
6	Discussione e Sviluppi Futuri	50
7	Conclusioni	51
7.1	Conclusioni	51
7.2	Lavori futuri	51
	Bibliografia	53

Elenco delle figure

Elenco delle tabelle

Tabella 3.1	Proprietà APIGuard Assurance per dominio.	4
Tabella 3.2	Domini di Assurance di APIGuard Assurance.	13
Tabella 3.3	Box Gradient applicato in APIGuard Assurance (D3.P3).	15
Tabella 4.1	Fasi della pipeline di assessment (1-2-4 bloccanti / 5-6-7 non-bloccanti).	18
Tabella 4.2	Gerarchia delle eccezioni custom (1-4 interrompono start-up; 5-6 recuperate localmente).	36
Tabella 5.1	Componenti del testbed e versioni al Milestone 1.	38
Tabella 5.2	Analisi statica della codebase APIGuard Assurance v0.1.0	41
Tabella 5.3	Verdetto release-readiness al Milestone 1.	43
Tabella 5.4	KPI di idempotenza su due run indipendenti.	43
Tabella 5.5	Status e finding count per test.	44
Tabella 5.6	Verifica live del teardown post-run.	45
Tabella 5.7	Metriche di sistema sui due run.	46
Tabella 5.8	Topologia del Directed Acyclic Graph (DAG) a Milestone 1.	48

Elenco dei codici

Capitolo 1

Introduzione

1.1 Motivazione e gaps

1.2 Obiettivi

1.3 Struttura della tesi

Capitolo 2

Background e Stato dell'Arte

2.1 Esempio di sezione

Come discusso in letteratura [1], ...

Capitolo 3

Metodologia, Architettura e Principi di Design

APIGuard Assurance è un framework di security assessment automatizzato per Application Programming Interface (API) REST esposte tramite gateway, progettato per essere applicazione-agnostico, contract-driven e deterministicamente riproducibile. Nasce dalla necessità di superare la frammentazione degli approcci manuali e target-specifici: strumenti che richiedono di essere riscritti per ogni nuovo target, che producono risultati non confrontabili tra esecuzioni, e che non possono essere integrati in processi di verifica continuativi. Il suo obiettivo è produrre un assessment sistematico, tracciabile e ripetibile, applicabile a qualsiasi API REST documentata senza modifiche al codice.

La presentazione di *APIGuard Assurance* segue un approccio *top-down*: dai principi generali di sistema che ne definiscono il perimetro fino alle scelte implementative che li concretizzano. Architettura e implementazione sono trattate in capitoli distinti perché operano su livelli di astrazione distinti. Il Capitolo 4 mostra come i principi descritti vengono implementati.

Il design di *APIGuard Assurance* è formalizzato in 39 proprietà architetture distribuite su sette categorie; la Tabella 3.1 le raccoglie come mappa di riferimento.¹

3.1 Principi Fondamentali: il Perché delle Scelte Architetture

In coerenza con la progressione *top-down* dichiarata, l'analisi ha inizio dal livello di massima astrazione: le decisioni che definiscono il carattere del tool prima ancora di

¹In grassetto le proprietà trattate in questo capitolo.

Dominio	Proprietà
D1: Architettura & Design	D1.P1, D1.P2, D1.P3 , D1.P4, D1.P5, D1.P6
D2: Estensibilità & Manutenibilità	D2.P1, D2.P2 , D2.P3, D2.P4, D2.P5, D2.P6, D2.P7, D2.P8
D3: Config & Riproducibilità	D3.P1, D3.P2, D3.P3, D3.P4, D3.P5
D4: Robustezza & Sicurezza	D4.P1 , D4.P2, D4.P3, D4.P4, D4.P5, D4.P6, D4.P7, D4.P8, D4.P9
D5: Qualità & Osservabilità	D5.P1, D5.P2, D5.P3, D5.P4, D5.P5
D6: CI/CD & DevEx	D6.P1, D6.P2, D6.P3
D7: Packaging & Deployment	D7.P1, D7.P2, D7.P3

Tabella 3.1: Proprietà APIGuard Assurance per dominio.

descrivere i componenti. Le quattro proprietà discusse in questa sezione costituiscono la struttura portante del sistema: le scelte architetturali trattate nelle sezioni successive si stratificano su di esse, ciascuna risolvendo una frizione specifica all'interno dei vincoli che queste proprietà fondamentali stabiliscono.

3.1.1 Agnosticismo Applicativo (D1.P1)

Quali target applicativi è possibile valutare senza modificare il codice del tool?

Tutta la conoscenza del target viene derivata a runtime da due sole sorgenti: il file `config.yaml`, che raccoglie i parametri operativi del deployment (descritti in dettaglio nella Sezione 3.1.3), e la specifica OpenAPI del target, che fornisce la topologia completa degli endpoint, la definizione dei parametri di path, query e header per ciascuna operazione, gli schemi dei payload di richiesta e risposta, e i codici di stato attesi. Ne consegue un vincolo strutturale preciso: APIGuard Assurance non contiene nessun riferimento hardcoded a path, strutture dati o comportamenti di un'applicazione specifica, e qualsiasi API REST documentata è testabile senza modifiche al codice.

Un test che interroga la superficie di attacco derivata dalla specifica e non individua endpoint pertinenti alle proprie condizioni di esecuzione produce **SKIP**, lo stato previsto per uno scope legittimamente vuoto, distinto dall'**ERROR** che segnalerebbe un malfunzionamento interno al tool. Il principio si estende a qualsiasi requisito non soddisfatto: un tool esterno non installato, la Admin API del gateway non raggiungibile, un insieme di endpoint privi di seed configurato; ciascuna di queste condizioni produce **SKIP** sui test che ne dipendono e lascia proseguire gli altri, a meno che il risultato non costituisca una violazione confermata di un controllo perimetrale fondamentale (come l'assenza totale di autenticazione su tutti gli endpoint esposti), caso in cui la pipeline può essere interrotta esplicitamente tramite configurazione.

La differenza tra uno script target-specifico e un tool di assessment generale non è di complessità, ma di generalità: uno script con path e strutture dati scritti nel codice funziona su quel target e solo su quello (sostituita l'applicazione, va riscritto), mentre un tool che deriva tutta la propria conoscenza dalla specifica formale del target è applicabile a qualsiasi API REST documentata senza modifiche. Questa generalità è precisamente il confine che separa un artefatto di progetto da un contributo riutilizzabile.

Il costo accettato è la dipendenza da una specifica valida. Una specifica obsoleta o deliberatamente incompleta produce un assessment parziale senza che il sistema possa rilevarne l'incoerenza: il tool non ha modo di distinguere un'assenza di vulnerabilità da un'assenza di copertura, perché l'unico oracolo di cui dispone è la specifica stessa. Questa tensione costituisce il limite strutturale discusso nella Sezione ?? e si qualifica come problema aperto nel testing contract-driven, non come difetto correggibile con intervento locale.

3.1.2 La Specifica OpenAPI come Unico Oracolo Formale (D3.P2)

Quale strumento formale governa la costruzione delle richieste e la valutazione delle risposte nell'intera pipeline?

La specifica OpenAPI è l'unico vincolo contrattuale che governa l'intera pipeline: costruisce ogni richiesta nel rispetto dei tipi, dei formati e dei parametri dichiarati, delimita la superficie di attacco agli endpoint documentati e valuta ogni risposta rispetto a ciò che il contratto prevede per quella specifica operazione.

Un fuzzer che genera payload senza conoscere il contratto affronta un ostacolo strutturale prima ancora di raggiungere la logica applicativa: una quota sostanziale delle richieste viene respinta con HTTP 400 dal layer di validazione dell'input, perché i tipi, i formati e i vincoli enumerati nello schema non sono rispettati. Questo è il *combinatorial wall* discusso nel Capitolo 2: lo spazio di tutti i payload sintatticamente possibili è vastissimo, ma lo spazio dei payload strutturalmente validi rispetto al contratto è definito e navigabile. Usare la specifica come oracolo formale abbatte quella barriera: ogni richiesta è costruita nel rispetto dei vincoli dichiarati, spostando il punto di osservazione dalla sintassi alla semantica.

Ne derivano due capacità operativamente distinte rispetto al fuzzing cieco. In primo luogo, la superficie di attacco viene delimitata con precisione: gli endpoint documentati dalla specifica costituiscono l'insieme di ciò che il contratto dichiara raggiungibile, mentre qualsiasi endpoint risponda senza essere incluso in quella lista è, per definizione, una Shadow API. In secondo luogo, le risposte ricevute vengono valutate rispetto a ciò che il contratto prevede per quella richiesta, non rispetto a euristiche generiche: un campo

sensibile restituito in risposta a una richiesta autenticata è un finding solo se lo schema di risposta contrattuale non lo dichiarava atteso.

Anche questa proprietà porta con sé una dipendenza dalla qualità della specifica, già enunciata per D1.P1 (Agnosticismo Applicativo), ma con una conseguenza distinta. In D1.P1 il problema era di copertura: una specifica incompleta riduce gli endpoint osservabili senza che il tool possa rilevarlo. Qui il problema è di affidabilità dei verdeti: una specifica contrattualmente errata produce valutazioni delle risposte basate su un oracolo sbagliato, e il tool non ha modo di distinguere un contratto corretto da uno incorretto. Si tratta del paradosso del Ground Truth, discusso nella Sezione ?? come limite strutturale del paradigma contract-driven e non come anomalia specifica di questa implementazione.

3.1.3 Config-Driven Development (D3.P1)

Esiste un'unica fonte di verità per tutti i parametri che governano il comportamento del tool?

Stabilito che la conoscenza del target risiede nella specifica OpenAPI, rimane da definire il meccanismo che governa l'esecuzione. Tutti i parametri operativi risiedono nel file `config.yaml`, strutturato in aree funzionali con responsabilità distinte: la localizzazione del gateway e della specifica OpenAPI; i parametri di esecuzione globali, tra cui soglia di priorità minima, strategia di filtraggio dei test, timeout per singola esecuzione e formato del logging; il dizionario `path_seed`, che associa a ogni parametro di path parametrico un valore concreto presente sul target; le configurazioni specifiche per singolo test, organizzate per dominio; e i parametri degli strumenti di analisi esterni, con abilitazione per tool, versione attesa e opzioni di invocazione. Le credenziali di autenticazione, per loro natura non versionabili, risiedono in un file `.env` separato, il cui contenuto viene interpolato dal loader prima del parsing. Il codice sorgente non contiene nessun literal decisionale hardcoded: qualsiasi parametro che possa variare tra un'esecuzione e l'altra è dichiarato esplicitamente e validato prima dell'avvio della pipeline.

La separazione tra logica e configurazione assegna a ogni variazione di comportamento un'unica origine dichiarata: il file `config.yaml` attivo durante quella run. Se il comportamento del tool cambia tra un'esecuzione e l'altra, è perché è cambiata la configurazione; se la configurazione è identica, l'output è identico. Questa centralizzazione è anche la condizione che rende possibile la proprietà discussa nella sottosezione seguente: un sistema che legga valori da variabili d'ambiente non dichiarate, da file di stato impliciti o da sorgenti runtime non controllate non può garantire che due esecuzioni successive producano lo stesso risultato, indipendentemente dalla correttezza della logica di test. Fissare la configurazione è il primo passo necessario per fissare l'output.

3.1.4 Riproducibilità Deterministica (D3.P4)

Come viene garantito che l'assessment produca risultati confrontabili tra esecuzioni successive?

Lo stesso target, nella stessa configurazione, interrogato con lo stesso `config.yaml`, deve produrre risultati identici a ogni esecuzione. La riproducibilità è il presupposto che conferisce valore dimostrativo ai risultati del Capitolo 5: senza di essa, ogni run è un'osservazione singola non confrontabile, e l'assessment non è verificabile da terzi.

Il determinismo è ottenuto attraverso tre meccanismi distinti che si completano a vicenda:

- D3.P1 (Config-Driven Development): se tutti i parametri decisionali sono fissi, la pipeline non ha sorgenti di variazione endogene.
- Ordinamento lessicografico dei test all'interno di ogni batch del DAG: l'ordinamento topologico puro non garantisce stabilità tra test pronti allo stesso livello, e senza un criterio di tie-breaking esplicito la sequenza potrebbe variare tra esecuzioni; l'ordine lessicografico risolve questa ambiguità, garantendo che la sequenza sia sempre identica a parità di input.
- Struttura del DAG, trattata nella Sezione 3.2.2: un grafo aciclico orientato con risoluzione topologica deterministica produce sempre la stessa sequenza di batch, indipendentemente dall'ordine in cui i test vengono scoperti a runtime.

La garanzia di riproducibilità non riguarda ogni componente dell'output: timestamp e identificatori univoci dei singoli record di evidenza variano per natura tra esecuzioni. Ciò che deve essere invariante sono i contatori aggregati (PASS, FAIL, SKIP), i verdetti per singolo test e la distribuzione dei finding per dominio. La validazione empirica, condotta su due run indipendenti che hanno prodotto contatori aggregati byte-equivalenti (9 PASS / 7 FAIL / 2 SKIP / 0 ERROR / 98 finding, con `diff status per-test = 0` e delta wall-clock inferiore allo 0,3%), è discussa nella Sezione ??.

3.2 Architettura dell'Assessment Engine

Le proprietà discusse nella sezione precedente costituiscono la base del sistema. Le scelte di questa sezione si aggiungono ad esse, ciascuna rispondendo a un'esigenza architetturale specifica che le proprietà fondamentali lasciano aperta.

3.2.1 Flusso di Dipendenze Monodirezionale (D1.P2)

Il grafo delle dipendenze tra strati è strettamente aciclico e orientato in una sola direzione: il nucleo del sistema è importato dagli strati di integrazione, che sono importati dagli strati di test, che sono importati dall'orchestratore. Nessun modulo può importare da un livello superiore nella gerarchia. In termini operativi: modificare un test non può rompere il nucleo; aggiungere un connector non può influenzare l'orchestratore; l'intera base del nucleo può essere testata unitariamente senza istanziare alcun client HyperText Transfer Protocol (HTTP) né alcun test di sicurezza.

In assenza di un vincolo esplicito sulla direzione delle dipendenze, la pressione evolutiva naturale del codice tende verso cicli: un test che trova conveniente importare un'utility da un altro test, un connector che ha bisogno di un modello definito nell'orchestratore, e così via. La direzione è unidirezionale: un test può dipendere dal nucleo per accedere ai componenti condivisi, ma nessun modulo del nucleo può dipendere da un test. Ogni ciclo rende un modulo impossibile da isolare, e un modulo non isolabile non è testabile né sostituibile. Il vincolo è verificabile per ispezione del grafo di importazione: nessun modulo dello strato di test importa componenti degli strati superiori, e ogni strato dichiara esplicitamente nel proprio contratto le sole dipendenze che gli sono ammesse. I dettagli di come questo vincolo è garantito a livello di codice sono descritti nella Sezione 4.2.

La struttura a strati rende praticabile uno degli sviluppi futuri descritti nella Sezione ??: un modulo di test contribuito da terze parti viene scoperto automaticamente² senza che nessuna modifica al nucleo del sistema sia necessaria, perché il nucleo non contiene nessun riferimento ai test che lo utilizzano.

3.2.2 Scheduling Basato su DAG e Orchestrazione Deterministica (D2.P2)

L'ordine di esecuzione dei test è calcolato a partire dalle dipendenze che ogni test dichiara esplicitamente come parte del proprio contratto. Lo scheduler raccoglie queste dichiarazioni, costruisce un DAG orientato e risolve l'ordine topologico, producendo una sequenza di batch: gruppi di test il cui insieme di dipendenze è già stato soddisfatto. I batch sono sequenziali tra loro; all'interno di ogni batch i test sono privi di dipendenze reciproche. La traduzione di questo schema in codice, incluse le strutture dati e le librerie impiegate, è descritta nella Sezione 4.2.2.

²La scoperta automatica non è priva di vincoli: un modulo viene incluso nella pipeline solo se rispetta i contratti del tool, ovvero se dichiara tutti gli attributi obbligatori e si colloca nella directory attesa. Un modulo che non soddisfi questi requisiti viene ignorato silenziosamente oppure produce un errore di validazione.

Il problema che questa struttura risolve non è solo di ordinamento, ma di correttezza semantica. Un test che verifica la validità crittografica di un token JSON Web Token (JWT) presuppone che il meccanismo di autenticazione funzioni: se eseguito prima del test di autenticazione, il token non è disponibile nel contesto condiviso e il test fallirebbe per la ragione sbagliata. Senza una dichiarazione esplicita della dipendenza, la disponibilità del token è una preconditione implicita che dipende dall'ordine di esecuzione scelto: un cambio di scheduling la viola silenziosamente. Con la dipendenza dichiarata, il DAG verifica la preconditione a monte dell'esecuzione, e un token non disponibile diventa un errore rilevabile prima che un singolo test venga avviato.

Lo scheduler include due proprietà di correttezza strutturale:

- Rilevazione statica dei cicli: se il grafo delle dipendenze contiene un ciclo, l'errore viene segnalato prima che un singolo test venga eseguito, impedendo che stati inconsistenti si propaghino silenziosamente nella pipeline.
- Salvaguardia contro lo stallo: se il grafo risulta ancora attivo ma non riesce a estrarre nessun test pronto, il sistema emette un errore strutturato che elenca i test mai schedulati e interrompe l'esecuzione in modo controllato, fornendo all'operatore la diagnostica necessaria senza dover ricostruire lo stato interno del grafo.

L'ordinamento topologico puro non garantisce un ordine unico tra test pronti allo stesso livello: senza un criterio di tie-breaking³ esplicito la sequenza potrebbe variare tra esecuzioni. Lo scheduler risolve questa ambiguità ordinando lessicograficamente i test all'interno di ogni batch, convertendo la struttura parzialmente ordinata del grafo in una sequenza riproducibile, condizione necessaria per D3.P4 (Reproducibility).

La struttura a batch porta con sé una proprietà che la versione attuale non sfrutta: i test all'interno di uno stesso batch non hanno dipendenze reciproche e sono concettualmente parallelizzabili. Introdurre esecuzione parallela all'interno di un batch è una modifica localizzata allo scheduler, senza impatto sul design dei singoli test, a condizione che le implicazioni di concorrenza sullo stato condiviso vengano analizzate formalmente. Questa traiettoria è documentata come sviluppo futuro nella Sezione ??; la sequenzialità attuale è una scelta deliberata di scope, non un vincolo architetturale.

3.2.3 Split State e Immutabilità (D1.P3)

Durante l'esecuzione sequenziale dei test, due categorie di informazione coesistono nel processo: ciò che si sapeva del target prima che la pipeline iniziasse, e ciò che si scopre o si produce durante l'esecuzione. Trattarle con lo stesso oggetto mutabile introduce

³ *Tie-breaking*: regola di ordinamento applicata tra elementi a parità di livello topologico, per garantire una sequenza stabile e riproducibile tra esecuzioni.

una categoria di bug difficile da diagnosticare: un test che scrive accidentalmente sulla rappresentazione del target altera la vista condivisa da cui ogni verdetto dipende, senza che il sistema possa rilevarlo.

La soluzione è la separazione netta in due oggetti con caratteristiche opposte:

- **TargetContext** - immutabile per costruzione: viene popolato una volta sola prima che la pipeline inizi e qualsiasi tentativo di modifica produce un errore immediato nel punto in cui viene effettuato. Raccoglie tutto ciò che si conosce del target prima dell'esecuzione:
 - la specifica OpenAPI dereferenziata come mappa strutturata di endpoint, parametri e schemi;
 - i parametri di tuning specifici per ogni test ricavati dalla configurazione;
 - l'adapter del gateway iniettato prima dell'avvio della pipeline;
 - le credenziali, risolte dal loader a partire dal file `.env` e congelate nel **TargetContext** tramite la configurazione `frozen`.⁴
- **TestContext** - mutabile, raccoglie ciò che la pipeline produce. L'accesso è strutturato in tre canali tipizzati con responsabilità distinte, anziché in un dizionario ad accesso libero:
 - canale dei token: organizza le credenziali per ruolo, con accesso tramite un identificatore definito dal contratto del sistema, non tramite chiavi stringa arbitrarie;
 - canale di teardown: mantiene un registro Last In, First Out (LIFO) delle risorse create durante la run, garantendo che il cleanup segua l'ordine inverso della creazione, come discusso nella Sezione 4.5.3;
 - canale dei dati condivisi: consente a test in batch successivi di consumare risultati prodotti da batch precedenti, senza che tra i moduli vengano introdotte dipendenze di importazione.

L'implementazione concreta di questa separazione, inclusi i tipi e i meccanismi di accesso, è descritta nella Sezione 4.5.

⁴L'accesso avviene esclusivamente tramite i campi tipizzati del modello Pydantic costruito a runtime: non esiste un dizionario libero né accesso diretto ai valori grezzi.

Qualsiasi informazione scoperta durante l'esecuzione (endpoint presenti sul target ma non dichiarati nella specifica, token acquisiti, risorse create) appartiene per definizione allo stato dell'assessment e non alla conoscenza preliminare del target: viene quindi registrata nel contesto di esecuzione, non in quello del target immutabile. La separazione fa sì che la base informativa da cui ogni verdetto dipende non possa essere alterata durante la run, condizione necessaria perché D3.P4 (Reproducibility) sia dimostrabile empiricamente e non solo enunciabile come proprietà attesa.

3.2.4 Gateway-Agnostic Adapter (D1.P6)

D1.P1 stabilisce che la conoscenza del target applicativo deriva esclusivamente dalla specifica OpenAPI e dal `config.yaml`. La proprietà presentata in questa sezione estende lo stesso principio di agnosticismo a un livello distinto, lo strato di controllo del gateway: alcune garanzie di sicurezza, come la verifica che i timeout sui servizi upstream siano configurati, che il plugin di circuit breaking sia attivo o che le credenziali di servizio non siano hardcoded nelle variabili d'ambiente, non sono osservabili interrogando gli endpoint esposti ai client. Queste informazioni risiedono nel piano di configurazione amministrativo del gateway e richiedono un canale di accesso separato dai normali probe HTTP.

Il pattern scelto è un'interfaccia astratta di sola lettura verso il gateway: i test `WHITE_BOX` vi accedono esclusivamente tramite i metodi che questa interfaccia espone, senza contenere nessun riferimento al gateway specifico installato nel deployment. L'implementazione concreta viene selezionata a runtime in base alla configurazione e iniettata nel contesto del target prima che la pipeline abbia inizio. Se la Admin API del gateway non è configurata o non è raggiungibile, tutti i test `WHITE_BOX` transitano a `SKIP` con motivazione esplicita, senza errori a cascata. I dettagli dell'interfaccia e dell'implementazione Kong sono descritti nella Sezione 4.5.1.

Il vantaggio è che aggiungere supporto per un gateway diverso richiede di implementare una nuova classe concreta che soddisfi l'interfaccia, senza toccare nessuno dei test che la utilizzano: la logica di verifica non cambia, cambia solo il meccanismo con cui i dati di configurazione vengono recuperati. Il costo è che l'interfaccia può esprimere solo le verifiche comuni a tutti i gateway supportati: controlli che richiedano funzionalità vendor-specific non generalizzabili rimangono fuori dal perimetro dell'astrazione, e questo costituisce uno dei limiti strutturali discussi nella Sezione ??.

3.2.5 Streaming Evidence Store (D4.P1)

Un finding prodotto da un test acquista valore dimostrabile solo se accompagnato dall'evidenza HTTP che lo supporta: la request inviata, la response ricevuta, il timestamp e

gli identificatori di traccia. Senza di essa, l'esito del test è un'asserzione non verificabile da terzi, priva del requisito di non-repudiabilità che un audit trail tecnico richiede.

Il design originale dell'Evidence Store utilizzava un buffer a capacità fissa in memoria: i record più vecchi venivano rimossi silenziosamente quando il buffer si riempiva, generando riferimenti a record inesistenti nell'audit trail. Il risultato era un trail rotto senza che il sistema producesse nessun errore visibile. Il difetto era strutturale, non correggibile aumentando la capacità: qualsiasi limite fisso avrebbe trasformato la capacità in un parametro di tuning il cui valore corretto dipende da ciò che si vuole misurare, informazione non disponibile a priori.

L'Evidence Store scrive ogni record su file locali con flush immediato, senza buffering in memoria e senza dipendenze verso database o storage remoto: ogni record è permanente nel momento in cui viene prodotto e non esiste un limite superiore al numero di record per test. Il ciclo di vita dettagliato del componente è descritto nella Sezione 4.6.1. La scelta di operare esclusivamente su file locali è anche la condizione che abilita quanto descritto nella sottosezione seguente.

3.2.6 Assenza di Dipendenze di Stato Esterno (D3.P5)

APIGuard Assurance non ha dipendenze di stato esterno a runtime: tutto lo stato mutabile del processo risiede in memoria o in file locali, e le uniche connessioni di rete aperte durante un run sono quelle verso il target applicativo e, opzionalmente, verso la Admin API del gateway. Entrambe sono connessioni di test verso il sistema da valutare, non dipendenze infrastrutturali del tool. Un tool che richiede database, message broker o cache esterna trasferisce la propria affidabilità a servizi esterni: l'exit code al termine dell'esecuzione riflette lo stato dell'assessment solo se tutti quei servizi erano disponibili e corretti, una garanzia che il tool non può fornire autonomamente.

Nessun client di database, cache distribuita o Software Development Kit (SDK) di storage remoto figura tra le dipendenze obbligatorie del progetto: il deployment si riduce all'installazione del package e alla fornitura del file di configurazione. Questa assenza è anche la condizione che rende affidabile D6.P1: i semantic exit codes discussi nella Sezione ?? riflettono fedelmente lo stato dell'assessment perché nessun servizio accessorio può alterarli silenziosamente.

3.3 Domini di Sicurezza e Box Gradient Applicato

Le sezioni precedenti hanno definito le proprietà architetturali e strutturali su cui il sistema si fonda. Questa sezione definisce cosa il sistema misura e secondo quale logica

organizza le proprie misure: è il punto di convergenza tra la tassonomia delle minacce discussa nel Capitolo 2, dove le categorie di vulnerabilità sono state analizzate in quanto problema, e la struttura operativa del tool, dove quelle stesse categorie diventano il perimetro di garanzie che un assessment sistematico è tenuto a coprire.

3.3.1 Tassonomia degli Otto Domini di Assurance

La superficie di sicurezza di un'API esposta tramite gateway non è omogenea. Alcune garanzie sono verificabili senza nessuna conoscenza del sistema, interrogando semplicemente gli endpoint esposti senza credenziali; altre richiedono uno stato autenticato; altre ancora diventano accessibili solo attraverso il piano di configurazione amministrativo del gateway. La tassonomia adottata riconosce questa struttura di precondizioni, organizzando le garanzie in domini che rispecchiano categorie semantiche distinte di rischio.

La Tabella 3.2 li riporta con il loro titolo e l'ambito di verifica che ciascuno racchiude.

Dominio	Titolo	Ambito di verifica
D0	API Discovery	Inventario degli endpoint esposti; rilevazione di Shadow API non documentate nella specifica.
D1	Identity & Authentication	Meccanismi di riconoscimento del chiamante; robustezza del trasporto delle credenziali; ciclo di vita e revoca dei JWT.
D2	Authorization	Logiche di controllo degli accessi; difetti di segregazione orizzontale e verticale (Role-Based Access Control (RBAC), Broken Object Level Authorization (BOLA)).
D3	Data Integrity	Coerenza strutturale dei payload; implementazione delle firme crittografiche (Hash-based Message Authentication Code (HMAC)).
D4	Availability & Resilience	Difese contro l'esaurimento delle risorse; audit di circuit breaker e soglie di rate limiting.
D5	Observability	Qualità e tempestività della telemetria generata dal sistema. (<i>Milestone 2</i>)
D6	Configuration & Hardening	Indurimento dell'infrastruttura di gateway; policy di routing; header di sicurezza HTTP.
D7	Business Logic & Sensitive Flows	Falle semantiche complesse: elusione dei vincoli di processo, Server-Side Request Forgery (SSRF), race condition su operazioni critiche.

Tabella 3.2: Domini di Assurance di APIGuard Assurance.

La sequenza dei primi tre domini rispecchia una catena logica di precondizioni: non ha senso verificare l'autorizzazione (D2) senza avere prima verificato che l'autenticazione funzioni (D1), così come non ha senso verificare l'autenticazione senza conoscere la superficie di attacco (D0). Un assessment che verifichi D2 in assenza di certezze su D1 raccoglie evidenza su un sistema il cui comportamento in caso di autenticazione compromessa è ignoto. I domini D3–D7 non seguono la stessa dipendenza lineare: appartengono a famiglie semantiche distinte e possono essere valutati in parallelo, a condizione che le precondizioni di accesso richieste da ciascun test siano soddisfatte. Questa catena logica si riflette nelle dipendenze dichiarate dai singoli test che operano su questi domini, il cui meccanismo di risoluzione è descritto nella Sezione 3.2.2.

Domain-5 occupa una posizione particolare nella tassonomia: il dominio è architetturealmente predisposto nella struttura del sistema, ma i test corrispondenti sono pianificati per la Milestone 2. La numerazione è riservata intenzionalmente per preservare la coerenza della sequenza nelle versioni successive.

3.3.2 Il Box Gradient come Mapping dalle Precondizioni al Privilegio (D3.P3)

La classificazione Black/Grey/White Box del testing, nota nella letteratura come distinzione tra gradi di visibilità del sistema sotto test [rif. Sezione 2.X], viene derivata in APIGuard Assurance dalle precondizioni concrete che ogni garanzia impone al tester: non è una categoria assegnata retroattivamente ai test, ma il prodotto di ciò che ciascuna verifica richiede prima ancora di poter iniziare. Alcune garanzie sono fisicamente inaccessibili senza uno specifico livello di privilegio, e quella inaccessibilità è il criterio che determina la postura.

Il mapping adottato, formalizzato come proprietà D3.P3 e sintetizzato nella Tabella 3.3, correla la strategia operativa alle precondizioni che ogni classe di garanzie impone.

Il mapping adottato in questa implementazione correla la strategia alla priorità di rischio assegnata a ogni test: i test `BLACK_BOX` hanno tipicamente priorità P0 (critica), i test `GREY_BOX` priorità P1–P2, i test `WHITE_BOX` priorità P3. Questa correlazione è una scelta di design coerente con la natura dei controlli, non un vincolo imposto dal framework: la priorità di ogni test è configurabile e il mapping può deviare quando le precondizioni specifiche lo richiedono, come nel caso del test SSRF di Domain-7, classificato `GREY_BOX` con priorità P0.

Il livello di visibilità di ogni test è determinato da ciò che la verifica richiede per essere condotta correttamente. Un controllo perimetrale, ovvero la verifica che il gateway rifiuti richieste non autenticate, non presuppone nessuna conoscenza dell'infrastruttura interna: il risultato atteso è lo stesso indipendentemente da chi effettua la richiesta.

Strategia	Prerequisiti	Razionale
BLACK_BOX	Nessuno (solo accesso di rete)	Controlli perimetrali applicati dal gateway a qualsiasi interlocutore; simulazione di un attaccante esterno senza credenziali.
GREY_BOX	Token per ≥ 2 ruoli distinti	Garanzie che presuppongono stato autenticato con identità di ruolo distinte; inaccessibili senza credenziali valide.
WHITE_BOX	Accesso Admin API del gateway	Configurazione statica verificabile per ispezione diretta tramite piano di controllo del gateway.

Tabella 3.3: Box Gradient applicato in APIGuard Assurance (D3.P3).

Simulare un attaccante esterno senza credenziali è l'unico modo per ottenere evidenza non contaminata da privilegi impliciti, ed è per questo che i test **BLACK_BOX** sono la postura naturale per i controlli perimetrali.

Un controllo di autorizzazione RBAC, al contrario, richiede che il sistema si trovi in uno stato autenticato con almeno due identità di ruolo distinto. Senza quello stato il test non raggiunge nemmeno la logica che intende sondare: il gateway respinge la richiesta al livello di autenticazione, prima ancora che la logica di autorizzazione venga valutata, e il risultato osservato è indistinguibile da un corretto rifiuto di autenticazione. La strategia **GREY_BOX** è la risposta a questa preconditione.

I test **WHITE_BOX** occupano una posizione metodologicamente distinta. Alcune garanzie sono più efficacemente verificabili tramite ispezione diretta della configurazione del gateway che tramite probe empirici verso gli endpoint: la versione Transport Layer Security (TLS) supportata, la presenza e la parametrizzazione dei plugin di sicurezza, l'assenza di credenziali hardcoded nelle variabili d'ambiente. Un test empirico che cerchi di inferire questi parametri dall'esterno richiederebbe attacchi elaborati con esito incerto; un audit della configurazione li verifica in modo deterministico e riproducibile. È per questo che i test **WHITE_BOX** delegano la raccolta dei dati al componente di accesso al piano di controllo del gateway descritto nella Sezione 3.2.4, anziché effettuare probe sull'interfaccia pubblica.

Va rilevato che il mapping tra strategia e priorità è una raccomandazione strutturale, non un vincolo rigido imposto dal framework. Il codebase contiene deviazioni architetturealmente giustificate: il test di SSRF sul Domain-7 ha priorità P0 ma strategia **GREY_BOX**, perché la verifica di un endpoint che accetta Uniform Resource Locator (URL) controllabili dall'utente per effettuare richieste lato server richiede che l'endpoint sia raggiungibile con una richiesta autenticata. Il framework non vieta questa deviazione: la strategia appartiene al test, non al dominio, e la sua correttezza è una responsabilità

dello sviluppatore del test.

Ne deriva che il gradiente Black/Grey/White Box, applicato in questa forma, è il prodotto delle precondizioni informative e operative che ciascuna garanzia di sicurezza impone al tester: la tecnica segue il requisito, non viceversa. Questa distinzione ha conseguenze pratiche rilevanti per l'usabilità del framework in contesti industriali, dove la disponibilità delle credenziali e dell'accesso amministrativo varia da deployment a deployment e non può essere assunta come uniforme. La Sezione ?? discute come questa variabilità di privilegio si integri con le pipeline Continuous Integration / Continuous Deployment (CI/CD) senza richiedere configurazioni fisse. ““

Capitolo 4

Implementazione

Le 39 proprietà architetturali formalizzate nel Capitolo 3 trovano in questo capitolo la loro traduzione in codice Python. Non tutte ricevono lo stesso grado di trattamento: alcune strutturano interi componenti e vengono discusse in sezioni dedicate; altre emergono come conseguenza naturale delle scelte descritte e vengono citate nel contesto; altre ancora, di natura prettamente implementativa, compaiono nella tabella tassonomica del Capitolo 3 come riferimento. In ogni caso nessuna proprietà viene ignorata: il lettore che vuole rintracciare la realizzazione di una proprietà specifica può usare quella tabella come indice e poi cercare il locus nel codice indicato.

La distinzione rispetto al capitolo precedente è di livello di astrazione. Il Capitolo 3 descrive cosa fanno i componenti e perché sono stati progettati in quella forma; questo capitolo descrive come lo fanno, nominando classi, metodi e moduli specifici. I due livelli si richiamano con riferimenti espliciti: dove un componente è stato introdotto in termini architetturali nel Capitolo 3, questo capitolo aggiunge il dettaglio implementativo senza ripetere la motivazione.

4.1 La Pipeline di Esecuzione a Sette Fasi e il Core Engine

L'intero assessment si svolge in sette fasi sequenziali, orchestrate da `engine.py`. La struttura a fasi non è una convenzione organizzativa: riflette la dipendenza logica tra le operazioni e determina la semantica degli errori. Le fasi 1, 2 e 4 sono bloccanti, nel senso che un errore al loro interno interrompe lo startup prima che un singolo test venga eseguito; le fasi 5, 6 e 7 non bloccano mai, e i loro fallimenti vengono registrati nell'evidenza senza interrompere la pipeline. La Tabella 4.1 riporta il mapping completo tra fase, modulo responsabile e comportamento in caso di errore.

Fase	Nome	Modulo	Comportamento in caso di errore
1	Inizializzazione	<code>config/loader.py</code>	<code>ConfigurationError</code> — blocca lo startup
2	OpenAPI Discovery	<code>discovery/openapi.py</code>	<code>OpenAPILoadError</code> — blocca lo startup
3	Costruzione Contesti	<code>engine.py</code>	Nessuna eccezione attesa; <code>TargetContext</code> , <code>TestContext</code> e <code>EvidenceStore</code> vengono istanziati
4	Test Discovery e DAG	<code>tests/registry.py</code> , <code>core/dag.py</code>	<code>DAGCycleError</code> — blocca lo startup
5	Esecuzione	<code>engine.py</code>	Eccezioni per-test catturate internamente; il test produce <code>TestResult(ERROR)</code> e la pipeline prosegue
6	Teardown	<code>core/context.py</code>	<code>TeardownError</code> registrato come <code>WARNING</code> ; non invalida i risultati
7	Report	<code>report/renderers.py</code>	Output su <code>evidence.json</code> e <code>assessment_report.html</code>

Tabella 4.1: Fasi della pipeline di assessment (1-2-4 bloccanti / 5-6-7 non-bloccanti).

Il ruolo di `engine.py` in questa struttura è di orchestratore puro. Il file esegue le fasi nell'ordine corretto, passa i contesti da una fase all'altra, e non contiene nessuna logica di dominio: nessuna regola di sicurezza, nessun criterio di valutazione, nessuna conoscenza del formato della specifica OpenAPI. Questa separazione non è estetica. Un orchestratore che contiene logica di dominio diventa il punto in cui si concentrano le dipendenze, rendendo difficile testare i componenti in isolamento e impossibile sostituire una fase senza rischiare di rompere l'altra. La coerenza di `engine.py` con il principio del flusso monodirezionale di D1.P2 è verificabile per ispezione diretta: il file importa da `core/`, `discovery/`, `tests/` e `report/`, ma nessuno di questi moduli importa da `engine.py`.

4.1.1 Fail-Safe Error Isolation (D4.P3)

Il contratto che regola il rapporto tra `engine.py` e i test individuali è espresso da un'assenza: in `engine.py` non esiste nessun `try/except` attorno alla chiamata a `test.execute()`. Non si tratta di una dimenticanza. È la dichiarazione esplicita che la responsabilità di gestire le proprie eccezioni appartiene al test, non all'orchestratore.

Il contratto imposto da `BaseTest.execute()` stabilisce che il metodo deve sempre restituire un `TestResult` e non può mai propagare eccezioni verso l'esterno. La realizzazione concreta è un blocco `try/except Exception` all'interno di `execute()`, che in caso di eccezione imprevista costruisce e restituisce un `TestResult(status=ERROR, message=str(e))` con il traceback troncato a 500 caratteri tramite l'helper `_make_error()`. La pipeline riceve sempre un `TestResult` valido, indipendentemente da ciò che è successo internamente al test: nessun test può bloccare l'esecuzione degli altri.

La conseguenza operativa è misurabile. Un test che va in eccezione per un bug interno, per una risposta HTTP inattesa, o per un timeout non gestito produce un `TestResult(ERROR)` che viene registrato nell'evidenza con il suo traceback, e la pipeline prosegue con il test successivo nel batch. L'errore è visibile e diagnosticabile; la solidità della pipeline non dipende dalla correttezza di ogni singolo test.

4.1.2 Gerarchia delle Eccezioni con Phase Mapping (D4.P4)

Il meccanismo che rende possibile la distinzione tra errori bloccanti e non-bloccanti descritta in Tabella 4.1 è la gerarchia delle eccezioni custom. Tutte le eccezioni del tool sono sottoclassi di `ToolBaseError`, radice comune che permette al codice chiamante di catturare qualsiasi errore del tool con un singolo `except ToolBaseError` o di essere preciso con `except ConfigurationError`. La gerarchia conta 11 classi distribuite in 3 file, ciascuna con un mapping esplicito alla fase della pipeline in cui può essere sollevata. La Tabella 4.2 riporta la classificazione completa.

Due scelte meritano commento esplicito perché riflettono decisioni di design consapevoli, non vincoli tecnici.

La prima è la collocazione di `GatewayAdapterError` in `gateway/base.py` invece che in `exceptions.py`. La scelta segue il principio di locality: l'eccezione è definita nello stesso file dell'interfaccia che la solleva, rendendo il contratto dell'ABC autosufficiente. Un consumer di `BaseGatewayAdapter` trova tutto ciò che gli serve in un unico modulo, senza dover importare da `core/exceptions.py`.

La seconda è l'assenza deliberata di `ExternalToolNotFoundError`. Un tool esterno mancante non è una condizione di errore imprevista: è una condizione operativa at-

tesa, gestita a monte durante la Phase 4 tramite `is_available()` nel registry dei connector. Il test che dipende da un tool assente riceve uno stato `SKIP` con motivo esplicito, non un'eccezione. Introdurre `ExternalToolNotFoundError` avrebbe aggiunto una classe senza aggiungere semantica: la condizione è già rappresentata da `TestResult(status=SKIP)`.

4.2 Discovery Dinamico e Risoluzione del Grafo

Prima che un singolo test venga eseguito, l'engine deve sapere quali test esistono e in quale ordine eseguirli. Queste due responsabilità sono delegate a due componenti distinti: il `TestRegistry`, che scopre i test a runtime per ispezione del filesystem, e il `DAGScheduler`, che calcola l'ordine topologico a partire dalle dipendenze dichiarate. La separazione non è casuale: il registry opera su stringhe e classi Python, il scheduler opera su grafi di stringhe. Nessuno dei due conosce la logica interna dei test.

4.2.1 Discovery Dinamico via `pkgutil` (D2.P1)

Il `TestRegistry` in `src/tests/registry.py` non contiene nessuna lista hardcoded di test. Scopre le classi concrete di test attraverso tre fasi interne, denominate R1, R2 e R3 nella documentazione del modulo.

La Phase R1 è la scansione del filesystem tramite `pkgutil.walk_packages()`, che percorre ricorsivamente le directory `tests/domain_*` e importa tutti i moduli il cui nome rispetta il prefisso `test_`. La convenzione di nomenclatura è tecnica, non organizzativa: `pkgutil` usa il nome del file come discriminante, quindi un file chiamato diversamente non viene scoperto anche se implementa correttamente il contratto. Errori di importazione nei singoli moduli vengono catturati e registrati come `WARNING` strutturato senza interrompere la scansione degli altri.

La Phase R2 estrae le sottoclassi concrete di `BaseTest` dai moduli appena importati tramite `inspect.getmembers()`. Una classe è considerata concreta se non è `BaseTest` stessa, non è astratta, ed è effettivamente una sottoclasse. La Phase R3 applica i filtri: vengono trattenuti solo i test la cui `priority` rientra nelle priorità abilitate e la cui `strategy` rientra nelle strategie abilitate nel `config.yaml`.

L'`ExternalTestRegistry` in `src/external_tests/registry.py` segue la stessa logica con una fase aggiuntiva, denominata R4: dopo il filtraggio, raggruppa i test per `tool_name`, chiama `is_available()` una sola volta per gruppo, e inietta il connector condiviso nei test del gruppo se il tool è disponibile, oppure imposta `_skip_reason_from_registry`

su ogni test del gruppo se il tool è assente. Un solo **WARNING** nel log per tool mancante, indipendentemente da quanti test dipendono da quel tool.

La conseguenza per l'estensibilità è diretta. Aggiungere un nuovo test richiede creare un file nella directory di dominio corretta con il nome corretto e implementare il contratto di **BaseTest**: nessun altro file del sistema deve essere modificato per il discovery. Questa proprietà, formalizzata come D2.P1, è il prerequisito tecnico del plugin system menzionato come traiettoria futura nella Sezione ???: un package esterno che aggiunge la propria directory al `sys.path` e rispetta le convenzioni di nomenclatura viene scoperto automaticamente senza modifiche al core.

4.2.2 Scheduling Topologico e DAGScheduler (D2.P2)

Il **DAGScheduler** in `src/core/dag.py` riceve la lista ordinata di `test_id` prodotta dal registry e produce una lista ordinata di **ScheduledBatch**. Come illustrato nella Sezione 3.2.2 a livello architetturale, l'ordinamento è determinato dalle dipendenze dichiarate tramite la **ClassVar** `depends_on` di ogni test. Qui si descrive come quella logica è implementata.

Il componente centrale è `graphlib.TopologicalSorter` della libreria standard Python 3.9. Il **DAGScheduler** costruisce il grafo passando a `TopologicalSorter` un dizionario che mappa ogni `test_id` all'insieme dei suoi predecessori, chiama `prepare()` per verificare l'assenza di cicli, e poi drena il sorter livello per livello con il pattern `get_ready()` / `done()`. Ogni chiamata a `get_ready()` restituisce i nodi il cui insieme di dipendenze è già stato marcato come completato; `done(*ready_nodes)` sblocca i nodi che dipendono da quelli appena estratti. Ogni livello produce un **ScheduledBatch**.

ScheduledBatch è un `dataclass(frozen=True)` con due campi: `batch_index` (intero 0-based che riflette il livello topologico) e `test_ids` (lista di stringhe). L'immutabilità è deliberata: una volta costruito, un batch non deve essere modificato dall'engine durante l'esecuzione.

Il determinismo all'interno di ogni batch, requisito di D3.P4, viene garantito da una singola riga: `ready_nodes = sorted(sorter.get_ready())`. `graphlib` non garantisce l'ordine dei nodi estratti da un livello topologico; l'ordinamento lessicografico esplicito converte il non-determinismo del grafo in una sequenza riproducibile.

La stall detection gestisce uno scenario documentato ma teoricamente anomalo: `is_active()` restituisce `True` ma `get_ready()` non restituisce nodi, indicando che il sorter è in uno stato irrecuperabile nonostante l'assenza di cicli dichiarati. In questo caso il **DAGScheduler** calcola l'insieme dei `test_id` mai estratti, li registra in un **ERROR** strutturato con il cam-

po `unscheduled_test_ids`, e interrompe il drain. L'operatore dispone dell'elenco esatto dei test da ispezionare senza dover leggere un traceback generico.

4.2.3 Invarianti di `TestResult` a Livello di Tipo (D4.P9)

Ogni test produce un `TestResult`, il modello Pydantic che trasporta lo stato finale della verifica verso il report. Il modello potrebbe in linea teorica essere costruito con combinazioni semanticamente inconsistenti: un `PASS` con findings non vuoti, un `FAIL` senza evidenza, un `SKIP` senza motivazione. Ognuna di queste combinazioni produrrebbe un entry di report formalmente valido ma scientificamente privo di significato.

La soluzione adottata è un `@model_validator(mode="after")` in `src/core/models/results.py` che applica tre invarianti al momento della costruzione dell'oggetto, prima che raggiunga qualsiasi consumer:

1. `status=FAIL` richiede `findings` non vuoto. Un `FAIL` senza evidenza non è un `FAIL`: è un'asserzione.
2. `status=SKIP` richiede `skip_reason` non `None`. Ogni `SKIP` deve essere documentabile e auditabile.
3. `status=PASS` richiede `findings` vuoto. Un `PASS` non può coesistere con violazioni dichiarate.

Una violazione di uno degli invarianti solleva `pydantic.ValidationError` nel punto in cui l'helper `_make_pass()`, `_make_fail()`, o `_make_skip()` costruisce l'oggetto, ovvero all'interno del test che ha generato il risultato inconsistente. L'errore è immediatamente localizzato: il traceback punta al test responsabile, non a un punto remoto della pipeline di report.

Il contributo architetturale di questa scelta è la centralizzazione del contratto. Senza il validator, ogni test dovrebbe ricordare autonomamente di non costruire risultati inconsistenti: una convenzione distribuita su 18 implementazioni è una convenzione violabile. Con il validator, la correttezza è enforced da un singolo punto di verifica che non può essere aggirato senza modificare esplicitamente il modello.

4.3 Architettura dei Test Nativi e Contratto `BaseTest`

I test nativi sono le unità di verifica che comunicano direttamente con il target tramite `SecurityClient` e producono evidenza HTTP osservabile. Questa sezione ne descrive la

struttura interna: il contratto che ogni test deve rispettare per essere scoperto e orchestrato correttamente, il meccanismo di autenticazione condivisa, le regole di tracciabilità normativa, e la policy con cui i probe HTTP vengono interpretati. La Milestone 1 implementa test per i domini D0, D1, D2, D3, D4, D6 e D7; il Domain-5 (Observability) è architetturalmente predisposto e pianificato per la Milestone 2.

4.3.1 Gerarchia Duale `BaseTest` / `ExternalToolTest` (D1.P4)

Il sistema ospita due categorie di test con contratti distinti, e la distinzione non è organizzativa ma tecnica. I test nativi estendono `BaseTest` in `src/tests/base.py`; i test che integrano tool esterni estendono `ExternalToolTest` in `src/external_tests/base.py`. Le due classi base sono gerarchie parallele indipendenti: `ExternalToolTest` non eredita da `BaseTest`.

La separazione risolve un problema concreto di ereditarietà indesiderata. `BaseTest` espone metodi come `_log_transaction()` e `_probe_endpoint()`, che operano tramite `SecurityClient` per eseguire richieste HTTP contro il target. Un `ExternalToolTest` non usa `SecurityClient`: delega la raccolta dati al proprio connector. Ereditare quei metodi senza usarli avrebbe ampliato l'interfaccia pubblica di ogni test esterno con funzionalità non applicabili, rendendo il contratto meno preciso e la superficie di errore più ampia.

Dall'esterno, l'engine tratta le due gerarchie in modo quasi identico: entrambe producono un `TestResult`, vengono scoperte dal registry, e vengono schedulate dal `DAGScheduler`. L'unica differenza visibile in `engine.py` è un `isinstance` check al momento dell'invocazione, che seleziona la firma corretta di `execute()`. Il campo `TestResult.source` con tipo `Literal["native", "external"]` propaga la distinzione fino al report, dove i risultati nativi e quelli di tool specializzati vengono resi con sezioni visivamente distinte all'interno dello stesso dominio.

Il contratto imposto da `BaseTest` a ogni test concreto si articola in due livelli. Il primo è la dichiarazione obbligatoria di `ClassVar` che descrivono staticamente il test: `test_id` (identificatore univoco che corrisponde all'ID nella metodologia), `domain` (dominio di assurance 0-7), `priority` (P0-P3), `strategy` (`BLACK_BOX`, `GREY_BOX`, `WHITE_BOX`), `depends_on` (lista di `test_id` predecessori per il DAG), `cwe_id` e `tags` (riferimenti normativi). Il registry usa `has_required_metadata()` per scartare le classi con `ClassVar` incompleti prima ancora che vengano istanziate. Il secondo livello è l'implementazione di `execute(target, ctx, store)`, il metodo che non deve mai propagare eccezioni, come già discusso nella Sezione 4.1.1.

4.3.2 Auth Abstraction Layer e Idempotenza dei Token (D2.P5)

I test che operano in modalità `GREY_BOX` richiedono token validi per uno o più ruoli prima di poter raggiungere la logica applicativa che intendono sondare. La gestione di questi token è centralizzata nel modulo `src/tests/helpers/auth.py`, che espone la funzione `acquire_tokens()`.

Il meccanismo è idempotente per costruzione. Quando un test chiama `acquire_tokens(target, ctx)`, la funzione controlla prima se il `TestContext` contiene già token validi per i ruoli richiesti tramite `ctx.has_token(role)`. Solo se il token è assente esegue il login HTTP e scrive il risultato nel canale token del `TestContext` tramite `ctx.set_token(role, token)`. Con 18 test attivi in Milestone 1, di cui una parte significativa in modalità `GREY_BOX`, questa logica garantisce che il numero di login HTTP verso il target sia esattamente uguale al numero di ruoli distinti configurati, non al numero di test che li richiedono.

Il dispatcher in `auth.py` seleziona il meccanismo di autenticazione corretto in base al campo `auth_method` del `config.yaml`: nella configurazione attuale, il metodo supportato è l'autenticazione tramite API key di Forgejo. Aggiungere il supporto per un nuovo metodo, ad esempio Open Authorization 2.0 (OAuth 2.0) con client credentials, richiede una nuova implementazione del flusso di login e un ramo aggiuntivo nel dispatcher: nessun test esistente deve essere modificato, perché nessun test conosce il meccanismo di autenticazione sottostante, solo il ruolo che richiede.

4.3.3 Tracciabilità Normativa e Distinzione Finding/InfoNote (D5.P1, D5.P2)

Ogni test porta con sé, come `ClassVar`, i riferimenti normativi della garanzia che verifica: `cwe_id` identifica la classe di vulnerabilità nel Common Weakness Enumeration, e `tags` raccoglie le categorie Open Web Application Security Project (OWASP), i riferimenti National Institute of Standards and Technology (NIST) e gli Request for Comments (RFC) pertinenti. Questi metadati non sono annotazioni decorative: vengono propagati fino al modello `Finding` tramite il campo `references: list[str]`, e il report HTML li riporta a fianco di ogni risultato. Un analista che legge un finding può risalire direttamente alla sezione della metodologia, alla categoria OWASP API Security, e al Common Weakness Enumeration (CWE) senza consultare documentazione esterna.

La tracciabilità presuppone una distinzione semantica precisa tra due tipi di annotazione che un test può produrre. Un `Finding` è evidenza di una violazione di sicurezza: compare solo in risultati `FAIL`, viene contato nel totale complessivo dei finding, e il suo contenuto deve essere sufficiente a giustificare la valutazione negativa. Un `InfoNote` è un'annotazione informativa su un risultato `PASS`: non cambia lo status, non conta come violazione,

e serve a documentare contesto rilevante per l'analista senza alterare il verdetto. La distinzione non è estetica, è architetturale: i contatori di finding nel report aggregano esclusivamente oggetti **Finding**, rendendo il totale affidabile e non influenzabile da annotazioni informative.

Un caso d'uso concreto illustra il valore della distinzione. Il test 4.3 verifica la presenza di un meccanismo di circuit breaking sull'infrastruttura del gateway. Kong OSS, nella configurazione di Milestone 1, non implementa circuit breaking attivo ma espone health-check passivi che svolgono una funzione parzialmente equivalente. Il test registra **PASS** perché il requisito minimo è soddisfatto, e allega una **InfoNote** che documenta la differenza tra il compensating control osservato e il circuit breaking completo. L'analista ottiene un quadro preciso senza che il sistema produca né un falso **FAIL** né un **PASS** silenzioso che nasconde la sfumatura.

L'invariante che un **PASS** non può coesistere con **Finding** è enforced da D4.P9: il `model_validator` di `TestResult` garantisce la coerenza al momento della costruzione.

4.3.4 Safe HTTP Probing Policy e Oracle a Tre Esiti (D4.P7)

Ogni verifica di sicurezza che richiede l'invio di una richiesta HTTP verso un endpoint del target deve interpretare la risposta ricevuta in modo non ambiguo. Il problema non è banale: uno status **404 Not Found** su un endpoint parametrico come `/users/{id}/repos` può significare che il gateway ha rifiutato la richiesta per mancanza di autenticazione, che il path non esiste per il valore di `{id}` usato nel probe, o che l'endpoint è genuinamente non protetto ma il path era errato. Interpretare un **404** come **FAIL** in tutti i casi genera falsi positivi inaccettabili in un assessment che si propone come affidabile.

La policy adottata definisce un oracle a tre esiti applicato sistematicamente a ogni probe HTTP nei test nativi:

1. **ENFORCED**: la risposta è 401 o 403. Il gateway ha applicato un controllo di accesso. Il controllo funziona.
2. **BYPASSED**: la risposta è 2xx. Il gateway non ha applicato il controllo atteso. Il controllo è assente o mal configurato.
3. **INCONCLUSIVE**: qualsiasi altro status. Il risultato non è interpretabile con certezza sufficiente per produrre un **Finding**.

Solo un esito **BYPASSED** produce un **Finding** e contribuisce a un risultato **FAIL**. Un esito **INCONCLUSIVE** viene registrato nell'evidenza con il suo status effettivo, ma non altera il verdetto del test.

A questa logica di interpretazione si affianca la segregazione degli endpoint in due tier. Il Tier A comprende i path fissi della specifica, cioè endpoint senza parametri di path o con parametri che il sistema sa risolvere tramite seed configurato: la risposta è interpretabile perché il path è valido. Il Tier B comprende i path parametrici senza seed configurato, dove la risposta dipende dal valore del placeholder usato nel probe: questi endpoint producono tipicamente **INCONCLUSIVE** e vengono annotati come tali nell'evidenza, segnalando all'operatore che configurare il path seed risolverebbe l'ambiguità.

Un caso limite merita attenzione esplicita. I probe con metodo **DELETE** su endpoint parametrici presentano un rischio operativo: se il gateway è mal configurato e non applica autenticazione, una richiesta **DELETE** con un path valido potrebbe eliminare risorse reali sul target. La policy prevede un fallback sicuro: quando il path seed non è configurato per un endpoint **DELETE** parametrico, il probe viene eseguito con il valore letterale **"apiguard-probe"** come sostituto del parametro. Un path con quel valore non corrisponde a nessuna risorsa reale su alcun sistema, garantendo che il probe non causi effetti collaterali indesiderati anche in presenza di un gateway non autenticato.

4.4 Gerarchia dei Connettori e Integrazione Tool Esterni

I test della gerarchia **ExternalToolTest** delegano la raccolta di dati a tool specializzati: scanner di vulnerabilità, analizzatori TLS, fuzzer. Il layer dei connector è il punto in cui questa delega si traduce in codice: isola la logica di invocazione di ogni tool, normalizza il suo output in un modello comune, e si interpone tra il test e il dettaglio implementativo dell'integrazione. Senza questo layer, ogni test dovrebbe contenere la logica di subprocess management o di importazione della libreria, accoppiando la valutazione della garanzia di sicurezza alla meccanica dell'esecuzione del tool.

4.4.1 Gerarchia a Tre Livelli e Separazione Dati/Valutazione (D1.P5, D2.P3)

I connector seguono una gerarchia a tre livelli che segue lo stesso principio della dual test hierarchy: ogni classe eredita solo i meccanismi pertinenti al proprio pattern di integrazione.

BaseConnector in `src/connectors/base.py` è il primo livello: un Abstract Base Class (ABC) puro con un solo **ClassVar** (**TOOL_NAME: str**) e tre metodi astratti che costituiscono il contratto universale. **is_available()** verifica se il tool è utilizzabile nell'ambiente corrente; **get_version()** restituisce la versione del tool per fini di audit; **run(target_url, timeout_seconds)** esegue il tool e restituisce un **ConnectorResult**. Il parametro **timeout_seconds** è intenzionalmente privo di valore di default: ogni

chiamante deve fornirlo esplicitamente, letto da `config.yaml`, rendendo impossibile dimenticarlo.

Il secondo livello si biforca in due direzioni distinte a seconda della natura del tool. `BaseSubprocessConnector` serve i tool invocati come processo di sistema: fornisce implementazioni concrete di `is_available()` e `get_version()` basate su `subprocess`, e il metodo protetto `_run_subprocess()` che gestisce l'esecuzione, il timeout e il parsing dell'output. La ricerca del binario avviene tramite una cascata a tre canali, documentata come D7.P1: binario locale nella directory `tools/` del repository, poi `shutil.which()` per il path di sistema, infine una variabile d'ambiente per i deployment in cui il tool è esposto come microservizio HTTP. `BaseLibraryConnector` serve invece i tool Python-native: usa `importlib.util.find_spec()` per verificare la disponibilità della libreria senza invocare alcun `subprocess`, e legge la versione tramite `importlib.metadata`. Le implementazioni concrete, `NucleiConnector` e `TestsslConnector` per il primo ramo, `SslyzeConnector` per il secondo, si limitano a dichiarare le `ClassVar` specifiche del tool e implementare `run()`.

La separazione tra raccolta dati e valutazione è il principio che governa il contratto di `run()`. Il connector non decide se qualcosa è un `FAIL`: restituisce un `ConnectorResult`, modello Pydantic frozen che trasporta il campo `raw_output: dict[str, Any]`, `exit_code`, `execution_time_ms`, e `timed_out`. È l'`ExternalToolTest` che implementa `_evaluate()` e valuta il `ConnectorResult` contro il proprio oracle. Lo stesso `NucleiConnector` è quindi riutilizzabile da test con logiche di valutazione diverse: un test che usa nuclei per il shadow API discovery e un ipotetico secondo test che usa nuclei per injection scanning condividono la stessa istanza del connector senza che il connector conosca le differenze tra le due valutazioni.

4.4.2 Lifecycle dei Connector e Dependency Injection (D2.P4)

La Phase R4 dell'`ExternalTestRegistry`, introdotta nella Sezione 4.2.1, produce un effetto preciso sul lifecycle dei connector: `is_available()` viene chiamato esattamente una volta per tool, indipendentemente da quanti test dipendono da quel tool.

Il meccanismo si basa su due attributi di istanza di ogni `ExternalToolTest`. Il campo `_injected_connector` viene valorizzato dal registry con l'istanza del connector quando il tool è disponibile: i test del gruppo condividono la stessa istanza, costruita una sola volta. Il campo `_skip_reason_from_registry` viene valorizzato quando il tool è assente: ogni test del gruppo riceve la motivazione di skip già scritta, e il metodo `_run()` la controlla come prima operazione, restituendo `SKIP` immediatamente senza costruire il connector né interrogare il filesystem.

La conseguenza operativa misurabile è nel log di esecuzione. Con cinque test che dipen-

dono da nuclei e il binario assente, il log produce un solo **WARNING** del tipo *"nuclei not found — 5 tests will SKIP"*, non cinque entry identiche. In un assessment con molti tool e molti test, questa differenza rende il log leggibile invece che ridondante.

4.4.3 Classificazione A/B, Path Seed e Catalogo dei Payload (D2.P6, D2.P7, D2.P8)

I connector implementati in Milestone 1 sono classificati in due categorie che guidano le priorità di sviluppo e l'interpretazione dell'output dell'assessment. I connector di Categoria A sono prerequisiti per la completezza dei test **HYBRID**: la loro assenza produce **SKIP** su garanzie che altrimenti sarebbero parzialmente coperte. I connector di Categoria B espandono la copertura su garanzie già verificate da test nativi, senza essere necessari per l'assessment di base. Nella Milestone 1, **NucleiConnector** e **TestsslConnector/SslyzeConnector** sono entrambi Cat A: il primo perché copre shadow API discovery tramite template specializzati non replicabili con soli probe HTTP; i secondi perché l'analisi TLS richiede negoziazione di protocollo che supera le capacità di un client HTTP standard.

Il path seed system risolve un problema strutturale dell'agnosticismo applicativo. La specifica OpenAPI descrive endpoint con parametri di path come `/users/{owner}/repos/{repo}`: senza sapere quali valori concreti esistono sul target, il sistema non può costruire un path valido per il probe. Un path non valido produce tipicamente **404**, che l'oracle classifica **INCONCLUSIVE** per le ragioni descritte nella Sezione 4.3.4. Il file di seed, generato tramite il comando CLI `generate-seed` o configurato manualmente in `config.yaml`, fornisce valori concreti per i parametri di path dei target noti. Con il seed configurato per Forgejo, i path con `{owner}` e `{repo}` producono risposte interpretabili: **ENFORCED** se il gateway applica il controllo atteso, **BYPASSED** se non lo applica. La Sezione ?? riporta i dati quantitativi di questo effetto nell'assessment di Milestone 1.

I payload di attacco usati dai test nativi, come le sequenze di URL per il test **SSRF** e gli header malformati per i test di input validation, risiedono in moduli puri in `src/tests/data/`, separati dalla logica di valutazione. Aggiungere un nuovo vettore di attacco, ad esempio una nuova sequenza di redirect per bypass di validazione, richiede aggiungere una riga al catalogo: la logica del test che lo usa non deve essere modificata. In contesti enterprise dove i payload di attacco sono soggetti a un processo di approvazione separato dal codice applicativo, questa separazione non è un'opzione di stile ma un prerequisito operativo.

4.4.4 Isolamento della Dipendenza AGPL (D7.P2)

`sslyze`, la libreria Python per l'analisi TLS, è distribuita sotto licenza GNU Affero General Public License (AGPL) v3. Includere `sslyze` come dipendenza obbligatorio del core imporrebbe gli obblighi copyleft dell'AGPL a qualsiasi distribuzione di API-Guard Assurance, rendendo incompatibile la licenza Massachusetts Institute of Technology (MIT) del resto del codebase. La soluzione è dichiarare `sslyze` esclusivamente in `[project.optional-dependencies.sslyze]` del `pyproject.toml`: una dipendenza opzionale che richiede installazione esplicita.

Un'installazione standard tramite `pip install apiguard-assurance` non installa `sslyze` e non impone nessun obbligo copyleft. L'operatore che vuole abilitare il test `ext.1.5.sslyze` installa esplicitamente l'extra, accettando consapevolmente la licenza AGPL. A runtime, `SslyzeConnector` usa `importlib.util.find_spec("sslyze")` in `is_available()`: se la libreria non è installata, il metodo restituisce `False` e il test riceve `SKIP` con motivazione esplicita, senza errori di importazione. Il core del sistema non contiene nessun import condizionale di `sslyze`: l'isolamento è strutturale, non gestito con `try/except ImportError`.

Il pattern è replicabile per qualsiasi dipendenza con vincoli di licenza o peso computazionale elevato: l'architettura `BaseLibraryConnector` con discovery tramite `importlib` è progettata esattamente per questo uso, e l'aggiunta di un nuovo extra opzionale richiede solo una voce in `pyproject.toml` e una nuova sottoclasse di `BaseLibraryConnector`.

4.5 Implementazione del Gateway Adapter e Teardown LIFO

L'architettura del `BaseGatewayAdapter` è stata descritta nella Sezione 3.2.4 a livello di interfaccia e di motivazione. Questa sezione ne completa il quadro con l'implementazione concreta, descrive i tre livelli di degradazione controllata che il sistema applica quando parti dell'infrastruttura non sono disponibili, e illustra il meccanismo di cleanup delle risorse create durante l'assessment.

4.5.1 KongGatewayAdapter: Accesso Read-Only all'Admin API

`KongGatewayAdapter` in `src/core/gateway/kong.py` realizza l'interfaccia `BaseGatewayAdapter` interrogando la Admin API di Kong DB-less tramite richieste HTTP GET. Una scelta implementativa merita esplicitazione: il componente usa `httpx` direttamente, senza passare per `SecurityClient`. La motivazione è architetturale. Le chiamate alla Admin API

sono audit di configurazione dell'infrastruttura, non traffico di test verso il target applicativo; non devono apparire nell'EvidenceStore, non sono soggette alla policy di retry del `SecurityClient`, e appartengono a un confine di fiducia distinto da quello del API target. Usare un client `httpx` leggero e dedicato mantiene i due confini esplicitamente separati nel codice.

I metodi pubblici dell'adapter, `get_routes()`, `get_plugins()`, `get_services()`, `get_upstreams()`, e `get_plugin_by_name()`, delegano tutti a un metodo interno `_fetch_paginated()` che gestisce la paginazione della Admin API di Kong. Le risposte seguono il formato `{"data": [...], "next": "/path?offset=..."}`: il metodo segue il cursore `next` finché è non-nullo, aggregando i risultati in un'unica lista. In modalità DB-less tutti gli oggetti vengono restituiti in una singola pagina e `next` è sempre `null`, ma il meccanismo è corretto anche in caso di Kong con database in ambienti che lo prevedono.

Il metodo `check_connectivity()` viene invocato dall'engine durante la Phase 3 per verificare la raggiungibilità della Admin API prima di popolare `TargetContext.gateway`. Se la verifica fallisce, il campo rimane `None` e tutti i test `WHITE_BOX` transitano a `SKIP` tramite l'helper `_requires_admin_api()` di `BaseTest`. Qualsiasi errore sollevato dalle chiamate HTTP durante l'esecuzione dei test viene incapsulato in `GatewayAdapterError`, che il test cattura e converte in `TestResult(ERROR)` senza propagare l'eccezione all'engine.

4.5.2 Degradazione Controllata a Tre Livelli (D4.P5)

Un assessment di sicurezza eseguito in ambienti eterogenei incontra sistematicamente situazioni in cui parti dell'infrastruttura non sono disponibili: tool esterni non installati, Admin API non esposta, integrazione con tool esterni disabilitata per policy. La risposta corretta non è un errore a cascata che invalida l'intero run, ma una degradazione controllata che produce risultati parziali corretti con motivazioni esplicite.

Il sistema implementa tre livelli distinti di degradazione, ciascuno con un meccanismo specifico.

Il primo livello è il master switch `external_tools.enabled` in `config.yaml`. Quando impostato a `false`, l'`ExternalTestRegistry` restituisce una lista vuota senza scandagliare il filesystem né importare moduli: zero overhead, zero `SKIP` nel log, comportamento completamente silenzioso. Questo livello serve ambienti in cui la presenza di tool esterni è impossibile per policy di sicurezza o di deployment.

Il secondo livello gestisce i tool esterni assenti a livello di singolo tool. Come illustrato nella Sezione 4.4.2, la Phase R4 dell'`ExternalTestRegistry` imposta `_skip_reason_from_registry`

su ogni test del gruppo il cui tool non è disponibile: ogni test riceve **SKIP** con motivazione esplicita, un solo **WARNING** nel log per tool.

Il terzo livello gestisce l'assenza della Admin API del gateway. Se `gateway_adapter` non è configurato in `config.yaml` oppure se `check_connectivity()` fallisce, `TargetContext.gateway` rimane `None`. Ogni test **WHITE_BOX** chiama come prima operazione `self._requires_admin_api(target):` se il campo è `None`, restituisce immediatamente **SKIP** con il testo *"Admin API not configured"* senza eseguire nessuna logica di verifica. Il risultato dell'assessment in questo scenario copre i soli domini black-box e grey-box, con un numero di **SKIP** equivalente al numero di test white-box presenti nel set attivo.

La distinzione tra i tre livelli di **SKIP** è visibile nel report: il campo `skip_reason` di ogni **TestResult** porta la motivazione specifica, rendendo immediata la diagnosi di quale componente mancante ha ridotto la copertura dell'assessment.

4.5.3 Teardown Best-Effort in Ordine LIFO (D4.P6)

I test che raggiungono la logica applicativa del target spesso creano risorse necessarie alla verifica: token di autenticazione temporanei, utenti di test, risorse effimere. Lasciare queste risorse sul target al termine dell'assessment viola la proprietà di non-interferenza: un assessment successivo potrebbe incontrare uno stato alterato, e il target produttivo potrebbe accumulare risorse orfane nel tempo.

Il meccanismo di teardown è registrazione esplicita al momento della creazione. Ogni volta che un test effettua un'operazione che crea una risorsa persistente, chiama immediatamente `ctx.register_resource_for_teardown(method, path, headers)` passando l'endpoint di cancellazione. La registrazione avviene nella stessa transazione logica della creazione: se il test va in eccezione dopo la creazione ma prima di completare la registrazione, la risorsa potrebbe non essere rimossa. Questa condizione è documentata come limite noto e non è considerabile dai meccanismi di eccezione di D4.P3.

In Phase 6, l'engine chiama `ctx.drain_resources()` che restituisce le risorse registrate in ordine LIFO. L'ordine inverso rispetto alla creazione garantisce che risorse con dipendenze implicite vengano rimosse correttamente: una risorsa creata dopo un'altra e che dipende da essa viene eliminata per prima. Per ogni risorsa restituita, l'engine tenta la chiamata HTTP di cancellazione tramite **SecurityClient**. Un fallimento solleva **TeardownError**, che viene catturato e registrato come **WARNING** strutturato: il fallimento del teardown è un avvertimento operativo, non un'invalidazione scientifica dei risultati dell'assessment che lo ha preceduto. Come dimostrato nell'audit di Milestone 1 e discusso nella Sezione ??, zero risorse sono state rilevate come non rimosse su due run indipendenti.

4.5.4 Astrazione Trasparente dell'URL di Deployment (D7.P3)

Un assessment eseguito in sviluppo locale raggiunge il target all'URL `http://localhost:8080`; lo stesso assessment eseguito in un ambiente Docker Compose deve usare il nome del servizio Compose, ad esempio `http://api-gateway:8080`, per essere raggiungibile da un container che condivide la stessa rete. In assenza di un meccanismo di astrazione, ogni connector che costruisce URL di destinazione conterrebbe logica condizionale di deployment.

Il campo `effective_base_url` di `TargetContext` risolve questo problema una volta sola in Phase 3. L'engine legge `target.base_url` dal `config.yaml`, verifica se è presente la variabile d'ambiente `APIGUARD_TARGET_EFFECTIVE_URL` (usata per override nei deployment Docker), e scrive il risultato nel campo frozen di `TargetContext`. Da quel momento, tutti i connector usano `target.effective_base_url` tramite il metodo `target.effective_endpoint_base_url()`: zero logica condizionale distribuita nei connector, zero differenze tra i deployment visibili al di fuori di Phase 3.

4.6 Evidence Store e Sanitizzazione delle Credenziali

Ogni verifica di sicurezza produce due tipi di output: un verdetto, espresso come `TestResult`, e l'evidenza che lo sostiene, composta dalle transazioni HTTP effettuate e dai dati grezzi prodotti dai tool esterni. Il verdetto senza evidenza è un'asserzione; l'evidenza senza sanitizzazione è un rischio. Questa sezione descrive come `EvidenceStore` gestisce entrambi i problemi, e come i due deliverable finali dell'assessment, il file `evidence.json` e il report `assessment_report.html`, vengono prodotti e strutturati.

4.6.1 Ciclo di Vita dello Streaming Evidence Store (D4.P1)

La motivazione architetturale dell'`EvidenceStore` è stata introdotta nella Sezione 3.2.5: il design precedente basato su `deque(maxlen=100)` produceva audit trail rotti in silenzio. Qui si descrive l'implementazione concreta che ha sostituito quel design.

Il ciclo di vita dell'`EvidenceStore` si articola in quattro operazioni sincronizzate con le fasi dell'engine. In Phase 3, l'engine istanzia `EvidenceStore` e crea la directory temporanea `outputs/evidence_tmp/`. Prima di eseguire ogni test in Phase 5, l'engine chiama `store.begin_test(test_id)`: il metodo apre il file `outputs/evidence_tmp/{test_id}.jsonl` in modalità append e lo mantiene aperto per l'intera durata del test. Durante l'esecuzione, ogni chiamata a `store.log_transaction()` o `store.pin_artifact()` scrive immediatamente un oggetto JavaScript Object Notation (JSON) su una singola ri-

ga del file con `flush()` esplicito: zero buffering in memoria, zero rischio di perdita in caso di crash. Al termine del test, `store.end_test()` chiude il file. In Phase 7, `store.merge_and_finalize()` legge tutti i file `.jsonl` dalla directory temporanea, li ordina per `test_id` lessicografico per garantire un output deterministico, li concatena in un unico array JSON e li scrive nel file definitivo `outputs/evidence.json`. La directory temporanea viene poi rimossa.

Il vantaggio rispetto al design precedente è eliminare la classe di bug per costruzione. Con `deque(maxlen=100)`, il primo record di un test con più di 100 transazioni veniva silenziosamente rimosso dalla coda, generando `Finding.evidence_ref` che puntavano a record inesistenti nell'audit trail. Con il design corrente, ogni record scritto su disco è permanente. Il test 1.1 su Forgejo, che interroga oltre 100 endpoint documentati nella specifica OpenAPI, produce un file `1.1.jsonl` con un record per endpoint senza nessuna perdita: la dimensione dell'evidenza è limitata unicamente dalla costante nominata `RESPONSE_BODY_MAX_CHARS = 10 000` che tronca i body di risposta singoli, non il numero di record.

La proprietà di recuperabilità è un effetto collaterale dell'architettura. Se il processo va in crash tra Phase 5 e Phase 7, i file `.jsonl` rimangono su disco e sono leggibili manualmente con qualsiasi strumento JSON Lines (JSONL)-compatibile: l'evidenza parziale è recuperabile anche in assenza di un run completo.

4.6.2 Sanitizzazione Centralizzata delle Credenziali (D4.P2)

Un audit trail che contiene token di autenticazione, API key o password è esso stesso una vulnerabilità: il file `evidence.json` viene condiviso con il team di sicurezza, archiviato nei sistemi di ticketing, e potenzialmente incluso in repository. La sanitizzazione delle credenziali non può essere delegata ai singoli connector con l'aspettativa che ciascuno ricordi di redactare i campi sensibili: in un sistema estensibile da terze parti, quella convenzione non è verificabile staticamente e non è garantibile a lungo termine.

La risposta adottata è centralizzare la responsabilità in un unico punto. Il metodo `EvidenceStore._sanitize_artifact()` viene invocato automaticamente da `pin_artifact()` su qualsiasi dato prodotto da connector esterni prima che venga scritto su disco. Il connector che produce il dato non è tenuto a fare nulla: la garanzia è strutturale, non convenzionale.

La sanitizzazione opera con tre meccanismi sovrapposti che si completano a vicenda. Il primo è il key-pattern matching: una regex compilata a word-boundary controlla ogni chiave del dizionario ricorsivamente rispetto a 11 pattern (`token`, `password`, `api_key`, `apikey`, `authorization`, `bearer`, `secret`, `credential`, `auth`, `private_key`,

`access_token`). Qualsiasi chiave che corrisponde a uno di questi pattern vede il proprio valore sostituito con `[REDACTED]`, indipendentemente dal tipo del valore.

Il secondo meccanismo è la JWT value detection: la regex `_SANITIZE_JWT_PATTERN` riconosce la struttura a tre segmenti base64url separati da punto che caratterizza un JWT e redacta qualsiasi stringa con quella forma, indipendentemente dalla chiave che la contiene. Questo cattura i token che appaiono come valori di chiavi con nomi arbitrari, ad esempio in un campo `result` del raw output di un tool.

Il terzo meccanismo è l'header prefix matching: qualsiasi stringa che inizia con `"Bearer "`, `"Basic "`, `"Token "`, o `"token "` viene redactata, catturando i token che appaiono nel valore di header HTTP inclusi nei record di evidenza.

La tripla copertura garantisce che un token JWT leakato come valore di una chiave `"result"` arbitraria in un tool non noto al sistema venga comunque redactato, perché il secondo meccanismo opera sul valore indipendentemente dalla chiave. Un connector futuro contribuito da terze parti che dimentica di redactare i propri campi sensibili non crea una violazione nell'audit trail: la garanzia è nel punto di scrittura su disco, non nel punto di produzione del dato.

4.6.3 Dual Audit Trail e Report Domain-Centric (D5.P5)

I due deliverable finali dell'assessment assolvono funzioni complementari e si rivolgono a interlocutori distinti. `evidence.json` è il forensic trail tecnico: contiene ogni transazione HTTP, ogni artifact di tool esterno, ogni record di finding con la sua evidenza di supporto, in formato macchina nativamente ingestibile da log aggregator come Elasticsearch, Splunk o Loki. `assessment_report.html` è il deliverable operativo: generato da `report/render.py` tramite Jinja2, è leggibile da browser senza dipendenze esterne e riporta i risultati in linguaggio comprensibile a un team di sicurezza non specializzato sullo specifico tool.

La struttura del report è determinata dal modo in cui l'`EvidenceStore` aggrega i dati durante la Phase 7. Il `ReportBuilder` in `report/builder.py` raggruppa i `TestResult` per coppia (`domain`, `source`): per ogni dominio di assurance compaiono prima i risultati dei test nativi, con la loro evidenza HTTP diretta, poi i risultati dei test con tool specializzati, con il raw output del tool. Questa partizione non richiede sezioni fisicamente separate nel template: emerge naturalmente dall'aggregazione dei dati e dal campo `TestResult.source` che distingue `"native"` da `"external"`.

Il report include anche la transaction log di ogni test: la lista ordinata di tutte le transazioni HTTP effettuate durante l'esecuzione, incluse le richieste e le risposte con i body troncati alla costante `RESPONSE_BODY_MAX_CHARS`. Questa sezione, renderizzata

con lazy loading nel template HyperText Markup Language (HTML) per gestire report di grandi dimensioni, permette a un analista di ripercorrere l'intera sequenza di probe che ha prodotto un finding, riproducendo il ragionamento del tool senza dover rieseguire l'assessment.

Classe	Fase	File	Com- por- ta- men- to
ToolBaseError	—	exceptions.py	Ra- dice del- la ge- rar- chia
ConfigurationError	1	exceptions.py	Fa- tale — con- fig non va- lida
OpenAPILoadError	2	exceptions.py	Fa- tale — spec ir- rag- giun- gi- bile o in- va- lida
DAGCycleError	4	exceptions.py	Fa- tale — ci- clo nel- le di- pen- den- ze
AuthenticationSetupError	helper	exceptions.py	Fa- tale se le cre- den- ziali so- no in- va- lide
SecurityClientError	5	exceptions.py	Re- cu- pe- rata — pro- du-

Capitolo 5

Validazione sperimentale e risultati

I Capitoli 3 e 4 hanno descritto come APIGuard Assurance è stato progettato e realizzato. Questo capitolo verifica empiricamente che quelle scelte architetturali si comportino nel modo dichiarato quando il tool viene eseguito contro un target reale. La validazione si articola in quattro livelli analitici distinti: la topologia dell'ambiente di test (Sezione 5.1), la qualità ingegneristica del codebase misurata attraverso un audit di 73 verifiche (Sezione 5.2), la copertura funzionale e l'idempotenza dei risultati su due run indipendenti (Sezione 5.3), e infine il profilo di risorse computazionali in relazione alla topologia del DAG (Sezione 5.4).

Tutti i dati quantitativi di questo capitolo traggono origine dal Milestone 1 Pre-Release Audit, condotto il 2026-05-17 sulla versione 0.1.0 del tool con 73 verifiche indipendenti. Nessun risultato è stimato o interpolato: ogni numero citato ha un corrispondente punto di verifica nell'audit che lo ha prodotto. Questa tracciabilità è deliberata. Un capitolo di risultati senza riferimento alla procedura di misura che li ha generati è, dal punto di vista della riproducibilità, un capitolo di affermazioni.

5.1 Topologia del Testbed Sperimentale

La scelta dell'ambiente target non era vincolata a una specifica applicazione, ma a un insieme di requisiti derivati direttamente dalla matrice di copertura della suite. Serviva un'applicazione con una specifica Representational State Transfer (OpenAPI) nativa, non generata a posteriori, che esponesse endpoint protetti da autenticazione reale, supportasse almeno due livelli di privilegi distinti, e producesse comportamenti osservabili e stabili sufficienti a validare i Domini D0, D1, D2, D3, D4, D6 e D7 senza che il target dovesse essere modificato tra un run e l'altro. Forgejo 14.0.3, esposto attraverso Kong 3.9

in modalità DB-less, soddisfa tutti questi requisiti e aggiunge una caratteristica rilevante per la validazione dell'agnosticismo applicativo (discusso nella Sezione 3.1.1): il tool non contiene nessun riferimento hardcoded a Forgejo, e il medesimo file `config.yaml` che governa questo assessment è strutturalmente identico a quello che si userebbe su qualsiasi altra API REST documentata.

5.1.1 Architettura del Testbed

Il testbed è provisioned tramite un singolo file Docker Compose su host di sviluppo, senza hardware dedicato né virtualizzazione separata. La topologia comprende quattro servizi nel medesimo network `lab-net`: il database di persistenza (PostgreSQL 15-alpine), l'applicazione target (Forgejo 14), il gateway (Kong 3.9 DB-less) e un container `forgejo-setup` che esegue il provisioning degli account alla prima accensione e poi termina. La Tabella 5.1 riporta la matrice delle versioni fissata durante l'audit.

Componente	Versione	Ruolo nel testbed
<code>apiguard-assurance</code>	0.1.0	Tool sotto validazione
Python	3.12.3	Interprete del tool
Forgejo	14.0.3 (gitea-1.22.0)	Target applicativo: API REST con specifica OpenAPI nativa
Kong	3.9 DB-less	API Gateway: proxy, plugin, Admin API su porta 8001
PostgreSQL	15-alpine	Backend di persistenza Forgejo
<code>nuclei</code>	3.8.0	Tool esterno per Shadow API discovery (test <code>ext.0.1</code>)
<code>nuclei-templates</code>	10.4.3	Template pinned per riproducibilità dei scan
<code>testssl.sh</code>	3.2.3	Analisi TLS black-box (test <code>ext.1.5.testssl</code>)
<code>sslyze</code>	(libreria Python)	Analisi TLS via libreria (test <code>ext.1.5.sslyze</code>)

tati dell'analisi statica sul codebase AP

Tabella 5.1: Componenti del testbed e versioni al Milestone 1.

Kong espone due porte di proxy: la 8000 in HTTP semplice, usata esclusivamente dal test 1.5 per verificare il redirect verso Hypertext Transfer Protocol Secure (HTTPS), e

la 8443 con certificato self-signed generato localmente, che costituisce l'endpoint primario del tool. L'Admin API di Kong è accessibile sulla porta 8001 e rappresenta la sorgente dei dati per tutti i test di strategia `WHITE_BOX` dei Domini 3, 4 e 6. La configurazione Kong è interamente dichiarativa, contenuta in un file `kong.yml` montato in sola lettura: questo è il pattern DB-less che il box gradient della Sezione 3.3.2 identifica come prerequisito per il testing a visibilità piena sul gateway.

5.1.2 Configurazione Intenzionale per la Copertura dei Finding

Un aspetto metodologicamente rilevante della configurazione riguarda quali comportamenti del gateway sono stati deliberatamente non induriti al momento dei run di validazione. Un testbed completamente hardened produrrebbe una suite con tutti `PASS` e nessun `FAIL`: un risultato tautologico, non informativo. Che il tool sappia identificare le violazioni presenti in un ambiente realmente misconfigured è il dato di validazione funzionale più utile.

Di conseguenza, tre scelte di configurazione producono `FAIL` osservabili e attesi nell'assessment:

1. Il plugin `rate-limiting` di Kong è commentato nel file `kong.yml`. Nessun limite di velocità è attivo sul servizio Forgejo. Il test 4.1, che invia richieste in loop verso un endpoint protetto fino a ricevere `HTTP 429`, non riceve mai quel codice e registra `FAIL` con 2 finding. Si tratta di una condizione comune in ambienti di staging dove il rate limiting viene disattivato per non interferire con i test applicativi, e quindi rappresenta uno scenario realistico.
2. Forgejo espone token API e metadati di sessione attraverso alcuni endpoint documentati nella propria specifica OpenAPI senza applicare restrizioni di campo sulla risposta. Il test 1.1, che sonda ogni endpoint della specifica senza credenziali, registra 74 finding per gli endpoint che restituiscono `2xx` in assenza di autenticazione: il verdetto è `FAIL` con 74 finding nella categoria `BYPASSED`.
3. Il certificato TLS self-signed presenta configurazioni non ottimali rilevabili dall'analisi statica del protocollo. I test `ext.1.5.sslyze` e `ext.1.5.testssl` registrano rispettivamente 1 e 3 finding sulla configurazione TLS.

Al contrario, il plugin `response-transformer` è attivo e inietta gli header di sicurezza HTTP su tutte le risposte del servizio Forgejo: `Strict-Transport-Security`, `Content-Security-Policy`, `X-Content-Type-Options` e `X-Frame-Options`. Il test 0.1, che verifica la presenza di questi header, registra `PASS`. Il parametro `KONG_HEADERS=off` nella configurazione Kong rimuove l'header `Server: kong/version`, producendo `PASS` anche per il test 6.2 che verifica il fingerprinting del gateway. Il servizio `http-redirect-`

`service` produce un redirect 301 con header `Location` corretto per qualsiasi richiesta HTTP sulla porta 8000, consentendo al test 1.5 di verificare il redirect HTTP→HTTPS e registrare `PASS`. La combinazione di finding attesi e `PASS` attesi rende il testbed un banco di prova funzionalmente completo: il tool deve saper rilevare il primo gruppo e non produrre falsi positivi nel secondo.

5.1.3 Struttura RBAC e Provisioning degli Utenti

Forgejo viene provisioned con tre account tramite il container `forgejo-setup`: un amministratore (`thesis-admin`), un utente con privilegi ordinari (`user-a`), e un secondo utente senza privilegi specifici (`user-b`). Questa struttura a tre livelli è il minimo sufficiente per la matrice di test della Milestone 1. Il test 2.1 (`RBAC enforcement`, `GREY_BOX`, P2) richiede un token di `user-a` con cui tentare endpoint amministrativi: il gate di controllo accesso del gateway deve rispondere 403. I test 1.4 e 2.1 dipendono da un token valido ottenuto dal run del test 1.1, dipendenza espressa nel DAG come vincolo di Phase B (discusso nella Sezione 4.2 e verificato empiricamente nella Sezione 5.4).

Il provisioning è deterministico. Il container `forgejo-setup` usa l'interfaccia CLI di Forgejo con flag `--or-true`, così da non fallire se gli account esistono già da un avvio precedente. Questa scelta è l'implementazione concreta della proprietà di riproducibilità D3.P4: il testbed raggiunge lo stesso stato iniziale indipendentemente da quante volte il comando `docker compose up` viene eseguito.

5.2 Release-Readiness e Integrità Architettuale

Un tool che dichiara di essere uno strumento di assurance della sicurezza non può essere esente da quel medesimo rigore applicato alla propria implementazione. Questa considerazione non è retorica. Un codebase con errori di tipo non rilevati, dipendenze con Common Vulnerabilities and Exposures (CVE) note, o simboli pubblici privi di contratto documentato produce output il cui grado di affidabilità è difficilmente separabile dalla qualità del codice che lo genera. Prima di discutere cosa il tool ha trovato sul target, è quindi necessario stabilire su quali basi ingegneristiche poggia l'analisi.

L'audit di Milestone 1 ha eseguito 73 verifiche distribuite in 9 categorie. Le prime due, l'analisi statica (verifiche 1–5) e l'ingegneria di produzione (verifiche 37–56), misurano la qualità del codebase indipendentemente dall'output di assessment. I risultati sono riportati come prerequisito di credibilità, non come contributo autonomo.

5.2.1 Analisi Statica del Codebase

Su 90 file sorgente Python, la suite di analisi statica ha prodotto i seguenti esiti al momento dell'audit, riassunti nella Tabella 5.2.

Tool	Risultato	Dettaglio
ruff	0 errori	Ruleset E/W/F/I/N/UP/B/S/ANN: lint, import order, upgrade, security, annotation coverage
mypy	0 errori su 90 file	Modalità <code>strict = true</code> : <code>no-implicit-optional</code> , <code>disallow-untyped-defs</code> , <code>warn-return-any</code> e tutti i flag associati
bandit	0 finding severity medium+	3 Low (B404, S603×2 in <code>connectors/base.py</code>), tutti soppressi con <code># noqa: S603</code> documentati: invocazione subprocess controllata per design
vulture	0 finding	Confidence ≥80%: nessun simbolo pubblico inutilizzato. <code>execute</code> , <code>model_*</code> e <code>validator_*</code> esclusi per dispatching dinamico
pip-audit	0 CVE	Nessuna vulnerabilità nota nelle dipendenze dichiarate

Tabella 5.2: Analisi statica della codebase APIGuard Assurance v0.1.0

Il dato di `mypy` merita un approfondimento architetturale, perché trascende la semplice metrica di qualità del codice. La proprietà D6.P3 (Sezione 4.3.1

5.2.2 Dual-Layer Type Safety (D6.P3)

Zero errori su 90 file sorgente in modalità `strict` è un risultato, non un obiettivo. Il problema che questa proprietà risolve ha una struttura precisa: la verifica statica con `mypy` garantisce che il contratto tra i moduli, espresso tramite type hints, sia consistente lungo l'intera catena di dipendenze. Nessun punto di chiamata riceve un tipo incompatibile con quello dichiarato dal callee. Gli errori di interfaccia tra moduli vengono intercettati prima che il codice venga eseguito contro un target reale.

`Mypy` non copre però le system boundaries: il punto in cui dati esterni al codebase, un file `config.yaml` caricato da disco o il raw output di un connector, entrano nel sistema. Quel confine è presidiato da `Pydantic v2`. I `model_validator` e i `field_validator` dei modelli di configurazione applicano le invarianti al momento della costruzione dell'oggetto: un campo `timeout_seconds = None` con `enabled = True` nel `config.yaml` produce un `ConfigurationError` con il path del campo violato durante la Phase 1, non

un crash non gestito durante l'esecuzione dei test. La classe di bug “typecheck verde, runtime crash” è strutturalmente eliminata.

I due strati sono indipendenti e non ridondanti: ciascuno intercetta una categoria di errori che l'altro non vede. Il costo mantenuto è prossimo allo zero per chi aggiunge un test: type hints e modelli Pydantic con `frozen=True` sono già vincolati dalle Hard Rules del progetto. La safety non dipende dalla disciplina individuale di chi contribuisce.

5.2.3 Indicatori di Qualità dell'Ingegneria di Produzione

Oltre all'analisi statica, l'audit ha verificato una serie di proprietà ingegneristiche che contribuiscono alla riproducibilità e all'affidabilità operativa del tool. Tre meritano menzione esplicita per la loro rilevanza rispetto alle garanzie di assurance che il tool promette.

Il primo è la copertura della documentazione interna: 292 simboli pubblici su 292 analizzati tramite scansione Abstract Syntax Tree (AST) presentano una docstring di modulo o di classe (100%). In un tool estensibile per design, la documentazione dei contratti pubblici non è decorativa: è il meccanismo attraverso cui un nuovo test può capire cosa può chiamare e con quali aspettative di comportamento.

Il secondo è la riproducibilità della build. Due esecuzioni consecutive di `hatch build --target wheel` producono un artefatto con SHA-256 identico (451 261 byte). Una wheel non deterministica introduce una categoria di variabilità nel deployment che è metodologicamente inaccettabile per un tool che promette idempotenza sui propri risultati.

Il terzo è il cold install: in un ambiente virtuale Python pulito, senza alcuna dipendenza preinstallata, l'esecuzione di `pip install apiguard_assurance-0.1.0-py3-none-any.whl` seguita da `apiguard version` restituisce 0.1.0. Il tool è self-contained e non dipende da pacchetti installati nell'ambiente dell'esecutore.

Il verdetto complessivo dell'audit, sintetizzato nella Tabella 5.3, è: pronto per la difesa di tesi e per il deployment in ambiente chiuso su target con specifica OpenAPI documentata. Gli unici due item deferiti, il file `LICENSE` e il `CHANGELOG.md`, riguardano esclusivamente la distribuzione pubblica su PyPI e non hanno impatto sul comportamento operativo del tool.

Aspetto	Stato	Riferimento
Correttezza funzionale	✓	Analisi statica 100% verde su 90 file (verifiche 1–5)
Integrità architetturale	✓	18 <code>test_id</code> unici; DAG aciclico; gerarchia eccezioni 11 classi; dipendenze monodirezionali (verifiche 19–23)
Copertura documentazione	✓	100% docstring su 292 simboli pubblici; 265/265 Pydantic Field description (verifica 65)
Packaging e distribuzione	✓	Wheel 451 261 byte, SHA-256 identico su 2 build consecutive (verifica 67)
Cold install	✓	<code>pip install wheel</code> in fresh venv → <code>apiguard version</code> → 0.1.0 (verifica 64)
Resilienza di produzione	✓	Signal handling garantisce teardown Phase 6 su <code>KeyboardInterrupt</code> (verifica 47)
Item deferiti	2	LICENSE e CHANGELOG.md: solo per distribuzione PyPI pubblica, nessun impatto operativo

Tabella 5.3: Verdetto release-readiness al Milestone 1.

5.3 Copertura Funzionale e Idempotenza dell’Assessment

L’idempotenza non era una proprietà scontata da verificare. Test che creano risorse sul target, le interrogano, e poi le eliminano introducono dipendenze di stato che, se mal gestite, producono risultati diversi al secondo run: la prima esecuzione modifica il target, la seconda trova uno stato diverso. Che il sistema produca risultati identici su due run indipendenti è la dimostrazione empirica che la pipeline non lascia effetti collaterali osservabili, non un’affermazione di principio.

I due run sono stati eseguiti in sequenza nello stesso giorno contro lo stesso testbed, senza alcun intervento manuale tra l’uno e l’altro. La Tabella 5.4 riporta i Key Performance Indicator (KPI) aggregati; la Tabella 5.5 dettaglia lo status e il conteggio dei finding per ciascuno dei 18 test attivi.

KPI	Run 1	Run 2	Δ
PASS / FAIL / SKIP / ERROR	9 / 7 / 2 / 0	9 / 7 / 2 / 0	identico
Finding count totale	98	98	identico
<code>pass_rate_pct</code>	56.2%	56.2%	identico
<code>assessment_duration_seconds</code>	286.04 s	286.96 s	+0.32%
Exit code / label	1 / FAIL	1 / FAIL	identico

Tabella 5.4: KPI di idempotenza su due run indipendenti.

Test ID	Dominio	Tipo	Status	Finding
0.1	D0 — Discovery	Nativo	FAIL	16
0.2	D0 — Discovery	Nativo	FAIL	1
0.3	D0 — Discovery	Nativo	SKIP	0
1.1	D1 — Auth	Nativo	FAIL	74
1.4	D1 — Auth	Nativo	PASS	0
1.5	D1 — Auth	Nativo	PASS	0
1.6	D1 — Auth	Nativo	SKIP	0
2.1	D2 — Authorization	Nativo	PASS	0
3.3	D3 — Data Integrity	Nativo	PASS	0
4.1	D4 — Availability	Nativo	FAIL	2
4.2	D4 — Availability	Nativo	PASS	0
4.3	D4 — Availability	Nativo	PASS	0
6.2	D6 — Hardening	Nativo	PASS	0
6.4	D6 — Hardening	Nativo	PASS	0
7.2	D7 — Business Logic	Nativo	FAIL	1
ext.0.1.nuclei	D0 — Discovery	Esterno	PASS	0
ext.1.5.sslyze	D1 — Auth	Esterno	FAIL	1
ext.1.5.testssl	D1 — Auth	Esterno	FAIL	3
Totale				98

Tabella 5.5: Status e finding count per test.

5.3.1 Analisi dei Risultati per Dominio

La distribuzione dei finding non è uniforme. Il test 1.1, che sonda ogni endpoint della specifica OpenAPI di Forgejo in assenza di autenticazione, produce da solo 74 dei 98 finding totali: il 75.5% dell'intero conteggio. Questo non è un artefatto di un test mal calibrato. Forgejo espone una superficie documentata di oltre cento endpoint, e la configurazione Kong del testbed non applica autenticazione a molti di essi per design sperimentale (come discusso nella Sezione 5.1). Ciascun endpoint che risponde **2xx** a una richiesta non autenticata è un finding distinto nell'oracle a tre esiti della Safe HTTP Probing Policy (D4.P7): la cardinalità riflette la superficie reale del target, non un'amplificazione artificiale del contatore.

I due SKIP meritano una lettura separata. Il test 0.3 verifica che gli endpoint marcati **deprecated: true** nella specifica restituiscano **410 Gone** o portino un header **Sunset** valido (RFC 8594). La specifica OpenAPI di Forgejo 14.0.3 non dichiara nessun endpoint con questo marcatore: lo scope è legittimamente vuoto, e il test restituisce **SKIP** con motivo documentato. Il test 1.6 verifica la configurazione del session store tramite l'Admin API di Kong; nella configurazione del testbed, Kong non espone un session store

gestito esternamente, e la condizione di applicabilità non è soddisfatta. In entrambi i casi, SKIP non è un fallimento silenzioso: è la risposta semanticamente corretta all'assenza di condizioni di applicabilità, distinta dall'ERROR che indicherebbe un malfunzionamento della pipeline stessa.

I tre FAIL del Dominio 1 prodotti dai tool esterni riguardano la configurazione TLS. Il certificato self-signed del testbed presenta configurazioni subottimali rilevabili dall'analisi statica del protocollo: `ext.1.5.sslyze` rileva 1 finding, `ext.1.5.testssl` ne rileva 3. Questi risultati sono attesi e coerenti con l'uso di un certificato generato localmente per ambiente di laboratorio.

5.3.2 Idempotenza e Validazione del Teardown

Il delta sul wall-clock tra i due run è di 0.92 secondi su una durata media di circa 286 secondi, pari allo 0.32% della durata totale. Questo margine rientra interamente nel rumore delle latenze di rete HTTP verso il target e non è distinguibile dalla variabilità attesa di un sistema operativo in esecuzione su hardware condiviso. I contatori di finding, gli status per-test, il codice di uscita e il `pass_rate_pct` sono invece byte-identici: non vi è alcuna variazione nella parte deterministica del risultato.

L'idempotenza ha una preconditione concreta: il teardown deve restituire il target allo stato iniziale dopo ogni run, senza lasciare risorse residue che alterino i risultati dell'esecuzione successiva. La verifica live condotta dopo ciascuno dei due run ha confermato che la Phase 6 ha drenato il registro LIFO con 4 risorse transitorie e 0 fallimenti su entrambe le esecuzioni. La Tabella 5.6 riporta lo stato delle risorse osservabili sul target prima del primo run e dopo ciascuno dei due.

Risorsa sul target	Prima run 1	Dopo run 1	Dopo run 2	Verdetto
Token Forgejo thesis-admin contenenti "apiguard"	0	0	0	✓
Repository Forgejo con user-a prefisso "apiguard"	0	0	0	✓

Tabella 5.6: Verifica live del teardown post-run.

Zero risorse leakate su due run è la dimostrazione empirica che le proprietà D4.P6 (Best-Effort Teardown LIFO) e D3.P4 (Riproducibilità) si comportano nel modo dichiarato in

presenza di stato reale. Non è sufficiente che la logica di teardown sia corretta in teoria: deve produrre uno stato osservabile verificabile. Queste tabelle sono quella verifica.

5.4 Footprint Computazionale e Benchmarking del DAG

Il profilo di risorse di un tool di assessment non è una metrica accessoria. Un tool che satura la CPU, esaurisce la memoria o produce un output non ingestibile non è integrabile in una pipeline CI/CD senza hardware dedicato. I dati di questa sezione rispondono a una domanda precisa: su quale classe di macchina e in quale regime di esecuzione APIGuard Assurance è operativamente sostenibile?

La Tabella 5.7 riporta le metriche di sistema misurate sui due run dell’audit, ottenute tramite `/usr/bin/time -v` applicato al processo `apiguard run`.

Metrica	Run 1	Run 2
Wall-clock elapsed	290.14 s (4:50.14)	290.81 s (4:50.81)
User time	35.09 s	34.67 s
System time	41.51 s	41.62 s
CPU utilization (media)	26%	(simile)
Peak resident set size	287 MB	295 MB
Voluntary context switches	145.356	(simile)
Involuntary context switches	20.685	(simile)

Tabella 5.7: Metriche di sistema sui due run.

5.4.1 Regime di Esecuzione: IO-Bound, non CPU-Bound

Il dato più informativo nella tabella non è il wall-clock. È la relazione tra user time, system time e tempo totale. Su 290 s di esecuzione reale, il processo ha consumato circa 35 s di tempo CPU in user space e 41 s in kernel space: la somma è 76 s, il 26% del wall-clock. I restanti 214 s, il 74% del tempo totale, il processo li ha trascorsi in attesa: attesa delle risposte HTTP dal target, attesa del completamento dei processi esterni (nuclei, testssl.sh), attesa dell’I/O su disco per la scrittura dell’evidence store.

Da questo si ricava un’inferenza architetturale rilevante. Il collo di bottiglia non è l’interprete Python né il DAG scheduler, ma la latenza di rete verso il target. Ottimizzare il codice Python non sposterebbe la curva in misura apprezzabile. Ciò che sposta effettivamente il wall-clock è la parallelizzazione dei test che non hanno dipendenze reciproche,

traiettoria discussa nella Sezione ?? come sviluppo futuro di Milestone 2. In assenza di parallelizzazione, il profilo attuale è il lower bound dell'architettura sequenziale, non un'inefficienza correggibile con refactoring locale.

Il profilo di memoria è compatibile con l'integrazione CI/CD su runner standard. 287–295 MB di resident set size si collocano ampiamente entro i limiti delle configurazioni GitHub Actions e GitLab Continuous Integration (CI) di default (tipicamente 7 GB per runner condivisi), senza richiedere runner dedicati né configurazioni di memoria particolari. Il footprint su disco è 4.5 MB: `apiguard_report.json` da 2.26 MB, `assessment_report.html` da 2.0 MB e `evidence.json` da 254 KB. I tre file sono autosufficienti e non richiedono dipendenze esterne per essere consultati.

5.4.2 Topologia del DAG e Implicazioni sull'Ordinamento dell'Esecuzione

L'esecuzione sequenziale di 18 test in due fasi non è la struttura più semplice realizzabile. Sarebbe stato possibile eseguire tutti i test in un singolo batch senza DAG: meno codice, zero overhead di scheduling. Il problema che questa scelta avrebbe lasciato aperto è la correttezza dei risultati in presenza di dipendenze tra test.

Il test 1.4 (revoca del token) richiede un token valido prodotto da un login HTTP riuscito. Il test 2.1 (RBAC enforcement) richiede lo stesso token per tentare endpoint amministrativi con credenziali di utente ordinario. Entrambi dipendono dal fatto che il test 1.1 abbia già interrogato la superficie di attacco e che l'autenticazione abbia prodotto un token nel `TestContext`. Eseguire 1.4 o 2.1 in assenza di questo prerequisito non produce un `ERROR` in fase di bootstrap: produce risultati non deterministici, ovvero output che dipendono dall'ordine di esecuzione e non dall'effettivo comportamento del target.

La topologia del DAG all'audit di Milestone 1, riportata nella Figura concettuale della Tabella 5.8, risolve il problema alla radice: 16 test senza dipendenze formano la Phase A ed eseguono per primi; 1.4 e 2.1, che dichiarano `depends_on = ["1.1"]`, vengono schedulati in Phase B e non possono mai essere avviati prima che il test 1.1 abbia prodotto un risultato.

Due considerazioni emergono da questa topologia. La prima è che il DAG è sparso: su 18 nodi, solo 2 hanno dipendenze dichiarate, e queste convergono su un unico predecessore. L'overhead di scheduling introdotto dal `DAGScheduler` è quindi trascurabile rispetto al tempo di esecuzione dei test; la complessità del componente si giustifica non sul carico attuale ma sulla scalabilità futura, quando dipendenze tra domini distinti renderanno il grafo più denso.

Fase DAG	Cardinalità	Test
Phase A (nessuna dipendenza)	16 test	0.1, 0.2, 0.3, 1.1, 1.5, 1.6, 3.3, 4.1, 4.2, 4.3, 6.2, 6.4, 7.2, ext.0.1.nuclei, ext.1.5.testssl, ext.1.5.ssllyze
Phase B (depends_on = ["1.1"])	2 test	1.4, 2.1

Tabella 5.8: Topologia del DAG a Milestone 1.

La seconda considerazione riguarda la relazione tra topologia del DAG e footprint di memoria. Con 16 test in esecuzione sequenziale nella Phase A, il **EvidenceStore** mantiene aperto un file `.jsonl` per volta: il picco di memoria non è determinato dal numero di test paralleli ma dalla dimensione del singolo audit trail. I 287–295 MB misurati rappresentano il profilo reale di questa esecuzione, non una stima conservativa. La transizione verso l'esecuzione parallela dei test all'interno della stessa fase, prevista in Milestone 2, richiederà una rivalutazione di questo profilo: più file `.jsonl` aperti concorrentemente comportano un footprint proporzionalmente maggiore, da misurare prima di fissare i requisiti di memoria per deployment CI/CD.

Sintesi della Validazione

I quattro livelli di analisi di questo capitolo non sono autonomi: si stratificano in una gerarchia di precondizioni. Il testbed stabilisce le condizioni di osservazione. La release-readiness certifica che lo strumento di misura è affidabile. La copertura funzionale e l'idempotenza dimostrano che i risultati prodotti sono stabili e non contaminati da effetti collaterali. Il profilo computazionale ne qualifica l'integrabilità operativa. Rimuovere uno qualsiasi di questi livelli non impoverisce la validazione: la invalida, perché ciascuno è la condizione di interpretabilità del successivo.

Il risultato aggregato è leggibile in termini precisi. Su un target reale con una specifica OpenAPI nativa, APIGuard Assurance v0.1.0 ha eseguito 18 test distribuiti su 7 domini di sicurezza, prodotto 98 finding in 9 categorie distinte, restituito risultati byte-identici su due run indipendenti, drenato tutte le risorse transitorie senza residui, e completato l'intero assessment in circa 4 minuti e 50 secondi su hardware non dedicato. Nessuno di questi numeri è stimato: tutti tracciano a un punto di verifica nell'audit.

Ciò che la validazione empirica non può fare è pronunciarsi sui limiti del paradigma che il tool implementa. Un assessment corretto su una specifica errata produce verdeti formalmente validi ma sostanzialmente privi di significato. Un tool che astrae il gateway per garantire portabilità multi-vendor accetta per costruzione di non poter accedere ai

controlli vendor-specifici che non abbiano un equivalente generalizzabile. Queste non sono anomalie dell'implementazione: sono tensioni strutturali del contratto che API-Guard Assurance stringe con il proprio dominio applicativo. La loro analisi è l'oggetto del Capitolo ??.

Capitolo 6

Discussione e Sviluppi Futuri

6.1

Capitolo 7

Conclusioni

7.1 Conclusioni

7.2 Lavori futuri

Bibliografia

- [1] “OWASP Top Ten Web Application Security Risks | OWASP Foundation.” [Online]. Available: <https://owasp.org/www-project-top-ten/>